

**Universitatea
Transilvania
din Brașov**

**FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR**

PROIECT DE DIPLOMĂ

Conducător științific:

Profesor dr. ing. Nicula Dan

Absolvent:

Terțiu Maria-Alexandra

BRAȘOV, 2026

Departamentul de Electronică și Calculatoare
Programul de studii: Calculatoare

TERȚIU Maria-Alexandra

Sistem pentru recunoașterea cifrelor scrise
de mână implementată cu o rețea
neurală convoluțională accelerată pe
FPGA

Conducător științific:

Prof. dr. ing. NICULA Dan

Brașov, 2026

Cuprins

Cuprins

Lista de figuri, tabele și coduri sursă.....	5
Lista de acronime	8
1 Introducere	9
1.1 Contextul și importanța temei	9
1.2 Problema abordată.....	10
1.3 Soluții existente și limitări	10
1.4 Soluția propusă și contribuția lucrării	11
1.5 Scopul și obiectivele lucrării.....	12
1.6 Structura lucrării	13
2 Fundamentare teoretică.....	15
2.1 Inteligența artificială și viziunea computerizată	15
2.2 Rețele neuronale artificiale	15
2.3 Rețele neuronale convoluționale (CNN).....	16
2.3.1 Straturi de convoluție	17
2.3.2 Funcția ReLU	18
2.3.3 Max-pooling	19
2.3.4 Straturi fully-connected.....	20
2.4 Implementarea rețelelor neuronale pe FPGA.....	21
2.5 Platforma Zynq și transferul datelor prin AXI/DMA	22
2.6 Reprezentarea numerică în hardware	22
2.7 Sistemul de operare embedded PetaLinux	23
3 Arhitectura sistemului propus.....	24
3.1 Funcționalitățile sistemului propus	24
3.2 Arhitectura hardware-software	25
3.3 Fluxul de date în sistem	26
4 Implementarea aplicației software	28
4.1 Interfața aplicației	28
4.2 Preprocesarea imaginii	30
4.3 Separarea și normalizarea cifrelor.....	32
4.4 Trimiterea cifrelor către ZedBoard	34
5 Dezvoltarea și evaluarea modelului CNN în software.....	36
5.1 Setul de date și pregătirea imaginilor.....	36
5.2 Arhitectura rețelei neuronale.....	38
5.3 Antrenarea modelului.....	39
6 Implementarea acceleratorului hardware.....	41

6.1	Organizarea implementării în Vitis HLS	41
6.2	Analiza reprezentării fixed-point	43
6.3	Implementarea funcției de inferență.....	44
6.4	Interfațarea acceleratorului prin AXI stream	48
6.5	Optimizări HLS utilizate	49
7	Integrarea sistemului embedded	51
7.1	Integrarea acceleratorului CNN în sistemul hardware	51
7.2	Aplicația embedded fpga_server	52
7.3	Concluzia capitolului	56
8	Rezultate experimentale, concluzii și direcții viitoare.....	57
8.1	Testarea reprezentării numerice fixed-point	57
8.2	Testarea optimizărilor HLS.....	59
8.3	Compararea timpului de inferență CPU, GPU și FPGA	62
8.4	Verificarea funcționării sistemului integrat	63
8.4.1	Testarea transmiterii unei cifre către accelerator	64
8.4.2	Testarea unor cazuri particulare	66
8.4.3	Limitarea observată: cifre pe verticală	67
8.5	Concluzii	67
8.6	Direcții viitoare	68
9	Bibliografie	70
	Rezumat.....	71
	Abstract	72

LISTA DE FIGURI, TABELE ȘI CODURI SURSĂ

FIGURI

Figura 2.1. Structura unui neuron artificial

Figura 2.2. Exemplu de aplicare a unui kernel 3×3 asupra unei imagini de intrare 7×7 .

Figura 2.3 Graficul funcției ReLU.

Figura 2.4. Exemplu de aplicare a funcției ReLU asupra unei hărți de caracteristici

Figură 2.5. Exemplu de operație max-pooling 2×2 .

Figura 3.1. Arhitectura de ansamblu a sistemului propus.

Figura 3.2. Arhitectura hardware-software a sistemului embedded de pe ZedBoard.

Figura 3.3. Fluxul de date pentru recunoașterea unei cifre.

Figura 4.1. Interfața aplicației software în timpul rulării.

Figura 4.2. Fereastra de debug.

Figura 4.3. Comparatie între imaginea binară inițială și imaginea binară curățată.

Figura 4.4. Separarea și normalizarea cifrelor.

Figura 5.1. Exemple de imagini din setul de date MNIST [4].

Figura 5.2. Exemple de variații utilizate pentru augmentarea datelor.

Figura 5.3. Arhitectura rețelei CNN.

Figura 6.1. Fluxul de dezvoltare în Vitis HLS [5].

Figura 6.2. Reprezentarea fluxului de date între Flatten, FC1 și FC2.

Figura 7.1. Integrarea acceleratorului CNN în sistemul hardware Zynq.

Figura 7.2. Maparea adreselor pentru acceleratorul CNN și AXI DMA în Vivado.

Figura 7.3. Fluxul datelor prin aplicația fpga_server, AXI DMA și acceleratorul CNN.

Figura 8.1. Acuratețea variantelor fixed-point uniforme pe setul de test MNIST.

Figura 8.2. Raport HLS pentru varianta O0.

Figura 8.3. Raport HLS pentru varianta O1.

Figura 8.4. Raport HLS pentru varianta O2.

Figura 8.5. Raport HLS pentru varianta O3.

Figura 8.6. Compararea latenței pentru variantele de optimizare HLS.

Figura 8.7. Configurarea adresei IP pe placa ZedBoard.

Figura 8.8. Verificarea conexiunii dintre PC și ZedBoard prin comanda ping.

Figura 8.9. Pornirea serverului TCP pe placa ZedBoard.

Figura 8.10. Testarea sistemului pentru o singură cifră.

Figura 8.11. Mesaje afișate în terminal în timpul clasificării unei cifre.

Figura 8.12. Testarea sistemului pentru mai multe cifre.

Figura 8.13. Rezultatul concatenat afișat pentru mai multe cifre detectate.

Figura 8.14. Clasificarea aceleiași cifre scrise în două moduri diferite.

Figura 8.15. Clasificarea unei cifre scrise înclinat.

Figura 8.16. Limitare observată pentru cifre dispuse vertical.

TABELE

Tabelul 4.1. Rolul panourilor din interfața aplicației software.

Tabelul 4.2. Rolul imaginilor afișate în fereastra de debug.

Tabelul 4.3. Ordinea etapelor de preprocesare a imaginii.

Tabelul 4.4. Criterii de filtrare a zonelor detectate.

Tabelul 4.5. Normalizarea unei cifre detectate.

Tabelul 4.6. Formatul datelor transmise prin TCP/IP.

Tabelul 5.1. Setul de date utilizat.

Tabelul 5.2. Transformările aplicate pentru augmentarea datelor.

Tabelul 5.3. Arhitectura modelului CNN.

Tabelul 5.4. Parametrii utilizați la antrenarea modelului.

Tabelul 5.5. Rezultatele modelului CNN în format Float32.

Tabelul 6.1. Organizarea fișierelor proiectului HLS.

Tabelul 6.2. Parametrii tipului de date `ap_fixed`.

Tabelul 6.3. Fluxul de calcul implementat în funcția `cnn_forward`.

Tabelul 6.4. Variante de transfer prin AXI Stream.

Tabelul 7.1. Registre principale utilizate pentru controlul AXI DMA.

Tabelul 7.2. Formatul datelor utilizate în transferul PC–ZedBoard–accelerator.

Tabelul 8.1. Acuratețea variantelor fixed-point uniforme pe setul de test MNIST.

Tabelul 8.2. Compararea variantelor fixed-point mixte testate în accelerator.

Tabelul 8.3. Compararea rezultatelor obținute la sinteză pentru variantele de optimizare.

Tabelul 8.4. Compararea timpului de inferență pe CPU, GPU și FPGA.

CODURI SURSĂ

Codul 6.1. Forma generală a tipului de date `ap_fixed` utilizat în Vitis HLS.

Codul 6.2. Tipurile fixed-point utilizate în varianta finală a acceleratorului.

Codul 6.3. Prototipul funcției `cnn_forward`.

Codul 6.4. Bufferele intermediare utilizate în funcția `cnn_forward`.

Codul 6.5. Exemple de funcții de indexare pentru greutatea Conv1 și FC1.

Codul 6.6. Funcțiile ReLU utilizate în implementarea CNN.

Codul 6.7. Implementarea stratului Conv1.

Codul 6.8. Implementarea operației MaxPool1.

Codul 6.9. Transformarea rezultatului Pool2 în vectorul flat.

Codul 6.10. Calculul scorurilor finale în stratul FC2.

Codul 6.11. Prototipul și directivele funcției `cnn_axis`.

Codul 6.12. Definirea tipului de pachet AXI Stream.

Codul 6.13. Calculul a patru produse pe iterație prin partiționare ciclică cu factor 4.

Codul 7.1. Extinderea valorilor `uint8_t` la `uint32_t` în bufferul de intrare.

Codul 7.2. Configurarea transferurilor DMA și pornirea acceleratorului CNN.

Codul 7.3. Funcția `argmax` aplicată pe scorurile din bufferul de ieșire.

LISTA DE ACRONIME

AI – Artificial Intelligence
ARM – Advanced RISC Machine
AXI – Advanced eXtensible Interface
BRAM – Block Random Access Memory
CLAHE – Contrast Limited Adaptive Histogram Equalization
CNN – Convolutional Neural Network
CPU – Central Processing Unit
DDR – Double Data Rate
DMA – Direct Memory Access
DSP – Digital Signal Processing
FC – Fully-Connected
FF – Flip-Flop
FPGA – Field Programmable Gate Array
GPU – Graphics Processing Unit
HLS – High-Level Synthesis
HP – High Performance
II – Initiation Interval
IP – Intellectual Property
LUT – Look-Up Table
MAC – Multiply-Accumulate
MM2S – Memory-Mapped to Stream
MNIST – Modified National Institute of Standards and Technology
OpenCV – Open Source Computer Vision Library
PC – Personal Computer
PL – Programmable Logic
PS – Processing System
ReLU – Rectified Linear Unit
ROI – Region of Interest
RTL – Register Transfer Level
S2MM – Stream to Memory-Mapped
SoC – System-on-Chip
TCP/IP – Transmission Control Protocol/Internet Protocol
VHDL – Very High Speed Integrated Circuit Hardware Description Language

1.1 CONTEXTUL ȘI IMPORTANȚA TEMEI

În ultimii ani, inteligența artificială a devenit un domeniu important în dezvoltarea aplicațiilor moderne, fiind utilizată pentru analiza datelor, recunoașterea unor modele și automatizarea proceselor de decizie. O ramură importantă a acestui domeniu este viziunea computerizată, care permite sistemelor de calcul să proceseze și să interpreteze imagini. Prin intermediul acesteia, un sistem poate extrage informații relevante dintr-o imagine și poate realiza operații precum clasificarea, detecția sau recunoașterea unor obiecte și forme [1].

Clasificarea imaginilor reprezintă una dintre aplicațiile principale ale viziunii computerizate. Aceasta presupune încadrarea unei imagini într-o clasă cunoscută, pe baza caracteristicilor vizuale identificate. Recunoașterea cifrelor scrise de mână este un exemplu relevant al acestei probleme, deoarece implică identificarea unor forme vizuale care pot varia semnificativ de la un utilizator la altul. O astfel de aplicație poate fi utilă în situații precum digitalizarea formularelor, recunoașterea codurilor numerice scrise manual, validarea datelor completate pe hârtie sau aplicații educaționale care interpretează răspunsuri scrise de utilizator. Pentru astfel de probleme, rețelele neuronale convoluționale reprezintă o soluție eficientă, deoarece pot extrage automat caracteristici vizuale din imagini, fără a fi necesară definirea manuală a tuturor regulilor de recunoaștere. Aceste modele sunt utilizate frecvent în aplicații de procesare a imaginilor datorită capacității lor de a identifica elemente precum linii, contururi și forme locale [2]. Pe lângă acuratețea clasificării, un aspect important îl reprezintă eficiența execuției. Rețelele neuronale convoluționale presupun un număr mare de operații matematice, în special înmulțiri și acumulări. În aplicațiile embedded, unde resursele hardware sunt limitate, timpul de procesare, consumul de resurse și modul de implementare devin aspecte importante.

În acest context, accelerarea hardware a rețelor neuronale pe platforme FPGA reprezintă o direcție relevantă de dezvoltare. FPGA-urile permit implementarea unor operații direct în hardware și exploatarea paralelismului, ceea ce le face potrivite pentru aplicații care necesită procesare eficientă [3]. Tema lucrării este importantă deoarece îmbină procesarea imaginilor, rețelele neuronale și proiectarea hardware, urmărind realizarea unui sistem embedded capabil să execute inferența într-un mod eficient.

1.2 PROBLEMA ABORDATĂ

Problema abordată în această lucrare constă în realizarea unui sistem capabil să recunoască cifre scrise de mână, corespunzătoare claselor de la 0 la 9. Recunoașterea automată a cifrelor presupune analizarea unei imagini de intrare și determinarea clasei corecte, chiar dacă forma cifrei poate varia în funcție de stilul de scriere, poziția în imagine, grosimea liniei sau calitatea capturii.

Pe lângă problema de clasificare, lucrarea tratează și problema implementării eficiente a rețelei neuronale într-un sistem embedded. Rețelele neuronale convoluționale presupun un număr mare de operații matematice, în special înmulțiri și acumulări, iar execuția acestora doar pe procesor poate conduce la timpi de procesare mai mari. Din acest motiv, este necesară analizarea unei metode de accelerare hardware a etapei de inferență.

Astfel, problema abordată nu se limitează la antrenarea unui model de inteligență artificială pentru recunoașterea cifrelor, ci include și integrarea acestuia într-un sistem funcțional. Sistemul trebuie să poată prelua o imagine, să o aducă într-un format compatibil cu rețeaua neuronală, să execute inferența pe acceleratorul hardware și să returneze rezultatul către aplicația software.

1.3 SOLUȚII EXISTENTE ȘI LIMITĂRI

Pentru implementarea rețelelor neuronale utilizate în clasificarea imaginilor există mai multe soluții, fiecare având avantaje și limitări. Alegerea platformei de implementare depinde de cerințele aplicației, de timpul de răspuns dorit, de consumul de energie, de cost și de resursele hardware disponibile.

O primă soluție este rularea modelului pe un procesor de uz general, adică pe CPU. Această abordare este des întâlnită, deoarece permite dezvoltarea rapidă a aplicațiilor și oferă flexibilitate în modificarea algoritmului. Modelele neuronale pot fi antrenate, testate și rulate în medii software folosind biblioteci precum TensorFlow sau Keras. Totuși, CPU-ul nu este optimizat pentru execuția masiv paralelă a operațiilor specifice rețelelor neuronale, cum sunt înmulțirile și acumulările repetate. Din acest motiv, execuția pe CPU poate deveni limitată în aplicațiile care necesită timp de răspuns redus.

O altă soluție este utilizarea plăcilor grafice, adică GPU-uri. Acestea sunt proiectate pentru execuția unui număr mare de operații în paralel și sunt folosite frecvent în antrenarea și rularea rețelelor neuronale. GPU-urile pot accelera semnificativ calculele față de un CPU, mai ales în cazul modelelor mari sau al procesării pe loturi de imagini. Cu toate acestea, ele pot avea un consum

mai ridicat de energie și nu sunt întotdeauna potrivite pentru sisteme embedded de dimensiuni reduse. În plus, integrarea unui GPU într-un sistem dedicat poate crește costul și complexitatea soluției.

O soluție alternativă este utilizarea circuitelor FPGA. Acestea permit implementarea unor operații direct în hardware și oferă posibilitatea exploatării paralelismului la nivel de circuit. Pentru rețelele neuronale convoluționale, FPGA-urile sunt utile deoarece operațiile repetitive, precum convoluțiile și înmulțirile acumulate, pot fi accelerate prin structuri hardware dedicate [3]. De asemenea, FPGA-urile pot fi potrivite pentru sisteme embedded, unde contează atât timpul de execuție, cât și utilizarea eficientă a resurselor.

Totuși, implementarea pe FPGA prezintă și limitări. Proiectarea hardware este mai complexă decât dezvoltarea unei aplicații software, iar resursele disponibile pe circuit sunt limitate. Modelul neuronal trebuie adaptat pentru implementarea hardware, ținând cont de reprezentarea numerică a datelor, memoria disponibilă, interfețele de comunicație și modul de transfer al datelor. Din acest motiv, este necesară o etapă suplimentară de conversie, optimizare și verificare a modelului.

În cadrul acestei lucrări, soluția aleasă este implementarea etapei de inferență pe o platformă FPGA, utilizând placa ZedBoard. Această alegere permite accelerarea calculelor specifice rețelei neuronale și integrarea modelului într-un sistem embedded. Astfel, soluția urmărește combinarea flexibilității părții software cu avantajele oferite de accelerarea hardware.

1.4 SOLUȚIA PROPUȘĂ ȘI CONTRIBUȚIA LUCRĂRII

Soluția propusă în această lucrare constă în realizarea unui sistem embedded pentru recunoașterea cifrelor scrise de mână, utilizând o rețea neuronală convoluțională accelerată hardware pe FPGA. Sistemul urmărește un flux complet de procesare, de la capturarea imaginii și prelucrarea acesteia în aplicația software, până la execuția inferenței pe acceleratorul hardware și afișarea rezultatului final.

Platforma utilizată pentru implementarea sistemului este ZedBoard, bazată pe arhitectura Zynq. Aceasta permite combinarea unui procesor ARM cu o zonă de logică programabilă, oferind posibilitatea separării sarcinilor între partea software și partea hardware. Procesorul este utilizat pentru control, comunicație și transferul datelor, iar logica programabilă este utilizată pentru accelerarea calculelor specifice rețelei neuronale.

În cadrul sistemului propus, modelul neuronal este antrenat în mediul software, iar etapa de inferență este implementată în hardware. Rețeaua neuronală primește o imagine prelucrată,

compatibilă cu formatul de intrare al modelului, și produce un set de scoruri corespunzătoare cifrelor de la 0 la 9. Cifra recunoscută este determinată pe baza scorului maxim.

Transferul datelor între procesor și accelerator se realizează prin mecanismul DMA, utilizând o interfață de tip AXI Stream. Prin această abordare, imaginea prelucrată poate fi transmisă către accelerator, iar rezultatele inferenței pot fi preluate de procesor pentru interpretare și afișare.

Contribuția lucrării constă în integrarea mai multor componente într-un sistem funcțional. Lucrarea nu tratează doar antrenarea unui model neuronal, ci și adaptarea acestuia pentru implementare hardware, realizarea acceleratorului în Vitis HLS, integrarea în sistemul Zynq și comunicarea cu aplicația software de pe PC.

Principalele contribuții sunt:

- realizarea unei aplicații software pentru capturarea, preprocesarea și transmiterea imaginilor;
- antrenarea și adaptarea unei rețele neuronale convoluționale pentru recunoașterea cifrelor scrise de mână;
- implementarea etapei de inferență sub forma unui accelerator hardware pe FPGA;
- integrarea acceleratorului într-un sistem embedded bazat pe ZedBoard;
- realizarea unui flux complet PC – ZedBoard – FPGA – PC, de la imaginea inițială până la rezultatul final.

1.5 SCOPUL ȘI OBIECTIVELE LUCRĂRII

Scopul lucrării este proiectarea, implementarea și validarea unui sistem embedded pentru recunoașterea cifrelor scrise de mână, utilizând o rețea neuronală convoluțională accelerată hardware pe FPGA. Lucrarea urmărește realizarea unei soluții funcționale, în care modelul neuronal este pregătit în mediul software și utilizat ulterior pentru inferență într-un sistem hardware-software integrat.

Pentru atingerea acestui scop, lucrarea are în vedere mai multe obiective. Primul obiectiv constă în realizarea unei aplicații software capabile să preia imaginea, să extragă cifra de interes și să o transforme într-un format compatibil cu intrarea rețelei neuronale. Această etapă este necesară pentru ca datele transmise către sistemul embedded să fie uniforme și potrivite pentru clasificare.

Al doilea obiectiv constă în antrenarea și evaluarea unei rețele neuronale convoluționale pentru clasificarea cifrelor de la 0 la 9. Modelul trebuie să fie suficient de compact pentru a putea

fi adaptat ulterior la implementarea hardware, dar și suficient de precis pentru recunoașterea imaginilor utilizate în testare.

Un alt obiectiv important este implementarea etapei de inferență sub forma unui accelerator hardware. Această etapă presupune transpunerea operațiilor rețelei neuronale într-o descriere compatibilă cu sinteza HLS și verificarea funcționării acceleratorului prin simulare și sinteză.

De asemenea, lucrarea urmărește integrarea acceleratorului într-un sistem embedded și realizarea comunicației dintre componentele software și hardware. La final, sistemul trebuie testat pentru a verifica funcționarea întregului flux, de la imaginea inițială până la obținerea cifrei recunoscute.

Obiectivele principale ale lucrării sunt:

- realizarea aplicației software pentru capturarea și preprocesarea imaginilor;
- dezvoltarea și testarea modelului CNN pentru recunoașterea cifrelor scrise de mână;
- adaptarea modelului pentru implementare hardware;
- implementarea acceleratorului de inferență utilizând Vitis HLS;
- integrarea acceleratorului în sistemul embedded bazat pe ZedBoard;
- realizarea transferului de date între componentele sistemului;
- testarea și validarea funcționării sistemului complet.

1.6 STRUCTURA LUCRĂRII

Lucrarea este organizată în opt capitole principale, urmate de bibliografie, rezumat, abstract și anexe.

Capitolul 1 prezintă contextul și importanța temei, problema abordată, soluțiile existente și limitările acestora, soluția propusă, contribuția lucrării, precum și scopul și obiectivele urmărite.

Capitolul 2 sintetizează noțiunile teoretice necesare pentru înțelegerea proiectului. Sunt prezentate concepte legate de inteligența artificială, viziunea computerizată, rețelele neuronale convoluționale, implementarea pe FPGA, platforma Zynq, transferul de date prin AXI Stream și DMA, reprezentarea numerică fixed-point și sistemul PetaLinux.

Capitolul 3 descrie arhitectura sistemului propus. Sunt prezentate componentele principale ale sistemului, rolul fiecărei componente și fluxul de date dintre aplicația software, placa ZedBoard și acceleratorul hardware.

Capitolul 4 prezintă implementarea aplicației software. Sunt descrise etapele de capturare a imaginii, preprocesare, extragere a cifrelor, conversie în formatul necesar rețelei neuronale și transmitere a datelor către sistemul embedded.

Capitolul 5 este dedicat modelului CNN și pregătirii acestuia pentru implementarea hardware. Sunt prezentate setul de date utilizat, arhitectura rețelei, procesul de antrenare, evaluarea modelului și exportul parametrilor utilizați în accelerator.

Capitolul 6 prezintă implementarea acceleratorului hardware în Vitis HLS. Sunt descrise organizarea proiectului HLS, reprezentarea fixed-point, funcția de inferență, interfațarea prin AXI Stream și optimizările aplicate.

Capitolul 7 descrie integrarea sistemului embedded pe platforma ZedBoard. Sunt prezentate conectarea acceleratorului în Vivado, utilizarea AXI DMA, aplicația fpga_server și fluxul datelor între PC, memoria DDR și accelerator.

Capitolul 8 prezintă rezultatele experimentale, testele realizate, concluziile lucrării și direcțiile viitoare de dezvoltare. Capitolul 9 conține bibliografia utilizată.

2 FUNDAMENTARE TEORETICĂ

2.1 INTELIGENȚA ARTIFICIALĂ ȘI VIZIUNEA COMPUTERIZATĂ

Inteligența artificială este un domeniu al informaticii care urmărește realizarea unor sisteme capabile să învețe din date și să rezolve sarcini precum clasificarea, recunoașterea imaginilor sau identificarea unor modele [2].

Viziunea computerizată reprezintă ramura inteligenței artificiale care se ocupă cu procesarea și interpretarea imaginilor de către un sistem de calcul [1]. În cadrul acestei lucrări, problema analizată este recunoașterea cifrelor scrise de mână, unde sistemul primește o imagine de intrare și trebuie să o încadreze într-una dintre cele 10 clase corespunzătoare cifrelor de la 0 la 9. Un set de date utilizat frecvent pentru această problemă este MNIST, alcătuit din imagini grayscale de dimensiune 28×28 pixeli [4].

Pentru clasificarea imaginilor sunt utilizate frecvent rețele neuronale artificiale. În cazul imaginilor, rețelele neuronale convoluționale sunt potrivite deoarece pot extrage automat trăsături locale, precum muchii, contururi și forme, care sunt apoi folosite pentru determinarea clasei imaginii [2]. Din acest motiv, în această lucrare este utilizată o rețea neuronală convoluțională pentru recunoașterea cifrelor scrise de mână.

2.2 REȚELE NEURONALE ARTIFICIALE

Rețelele neuronale artificiale sunt modele de calcul inspirate de modul în care neuronii biologici prelucrează informația. Un neuron artificial primește un set de valori de intrare, le înmulțește cu ponderi asociate, adaugă un termen de bias și produce o valoare de ieșire [2]. Relația generală poate fi exprimată astfel:

$$z = \sum_{i=1}^{\{n\}} w_i x_i + b \quad (1)$$

Unde:

- x_i reprezintă valorile de intrare;
- w_i reprezintă ponderile asociate intrărilor;
- b reprezintă termenul de bias;
- z reprezintă rezultatul intermediar obținut prin suma ponderată.

Într-o rețea neuronală, mai mulți neuroni sunt organizați în straturi. Un strat poate primi date de la stratul anterior și poate transmite rezultatele către stratul următor. Astfel, prin

conectarea mai multor neuroni și straturi, rețeaua poate învăța relații complexe între datele de intrare și rezultatul dorit.

În cadrul acestui proiect, valorile de intrare ale rețelei sunt obținute din imaginea prelucrată a cifrei. Pixelii imaginii sunt transformați în valori numerice, iar rețeaua folosește aceste valori pentru a determina cifra corespunzătoare. Parametrii rețelei, adică ponderile și bias-urile, sunt obținuți în etapa de antrenare.

Structura generală a unui neuron artificial este prezentată în Figura 2.1.

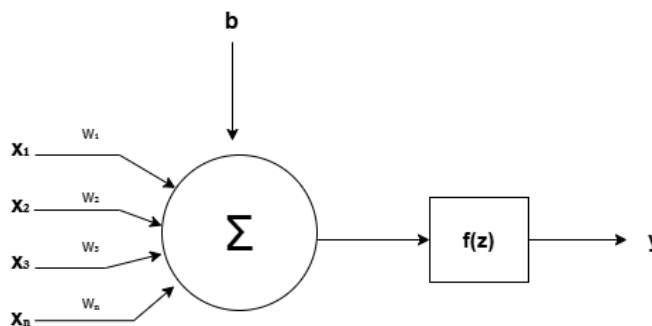


Figura 2.1. Structura unui neuron artificial.

Figura 2.1 evidențiază modul în care valorile de intrare sunt ponderate, însumate împreună cu termenul de bias și transformate într-o ieșire a neuronului. Acest principiu stă la baza straturilor utilizate în rețeaua neuronală implementată în proiect.

2.3 REȚELE NEURONALE CONVOLUȚIONALE (CNN)

Rețelele neuronale convoluționale, cunoscute și sub denumirea de CNN, reprezintă un tip de rețea neuronală utilizat în special pentru procesarea imaginilor. Acestea sunt potrivite pentru probleme de viziune computerizată deoarece păstrează structura spațială a imaginii și pot extrage caracteristici vizuale direct din datele de intrare. Spre deosebire de rețelele complet conectate, care tratează imaginea ca pe un vector de valori, rețelele convoluționale procesează imaginea local, pe regiuni mici.

O imagine digitală este formată din pixeli, iar fiecare pixel conține informații despre intensitatea luminoasă sau culoare. În cazul imaginilor grayscale, fiecare pixel este reprezentat printr-o singură valoare, iar în cazul imaginilor color, pixelul are de obicei trei valori, corespunzătoare canalelor roșu, verde și albastru. În cadrul acestui proiect sunt utilizate imagini grayscale de dimensiune 28x28 pixeli, asemănătoare celor din setul de date MNIST, deoarece problema abordată este recunoașterea cifrelor scrise de mână [4].

Arhitectura unei rețele CNN este alcătuită din mai multe tipuri de straturi, fiecare având un rol diferit în procesarea imaginii. Straturile de convoluție extrag caracteristici, funcția ReLU

introduce non-liniaritate, straturile de max-pooling reduc dimensiunea datelor, iar straturile fully-connected sunt utilizate pentru clasificarea finală [2]. Aceste componente vor fi prezentate separat în subsecțiunile următoare.

În cazul recunoașterii cifrelor scrise de mână, rețeaua primește o imagine de intrare și produce la ieșire un set de scoruri, câte unul pentru fiecare cifră de la 0 la 9. Cifra recunoscută este aleasă pe baza scorului cel mai mare. Astfel, rețeaua convoluțională combină extragerea automată a caracteristicilor vizuale cu etapa de clasificare, fiind potrivită pentru aplicația realizată în această lucrare [2].

În cadrul proiectului, rețeaua CNN este importantă și din punct de vedere hardware. Operațiile specifice, în special convoluțiile, presupun multe înmulțiri și acumulări, ceea ce poate duce la un timp de execuție mai mare pe CPU. Din acest motiv, implementarea rețelei pe FPGA permite accelerarea inferenței prin executarea unor calcule în paralel [3].

2.3.1 Straturi de convoluție

Stratul de convoluție este componenta care realizează extragerea caracteristicilor din imagine prin aplicarea unor filtre asupra datelor de intrare. Fiecare filtru, numit și kernel, este o matrice de dimensiuni reduse, de exemplu 3x3 sau 5x5, care se deplasează peste imagine și calculează valori noi pe baza regiunilor locale analizate. Rezultatul obținut în urma aplicării unui filtru se numește hartă de caracteristici sau feature map [2].

Modul de aplicare a unui kernel asupra unei imagini este ilustrat în Figura 2.2.

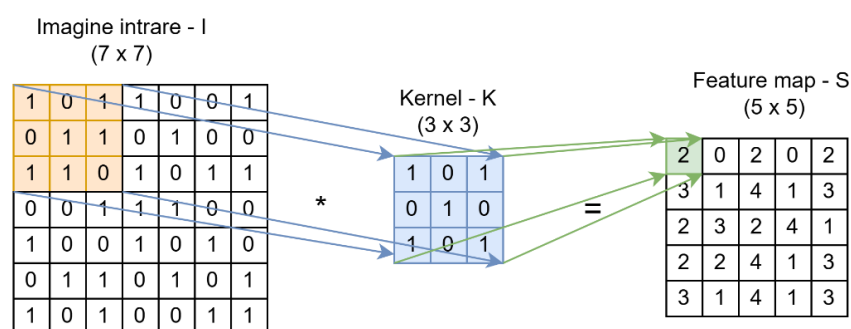


Figura 2.2. Exemplu de aplicare a unui kernel 3x3 asupra unei imagini de intrare 7x7.

În exemplul din Figura 2.2 este prezentată aplicarea unui kernel de dimensiune 3x3 asupra unei matrice de intrare 7x7. Pentru fiecare poziție a kernel-ului, valorile din zona selectată a imaginii sunt înmulțite cu valorile corespunzătoare din kernel, iar rezultatele sunt adunate. Valoarea obținută este plasată în matricea de ieșire, numită feature map. În exemplul prezentat, se consideră stride egal cu 1 și fără padding, astfel încât dimensiunea feature map-ului rezultat este 5x5.

Relația simplificată pentru calculul unei valori din feature map este:

$$S(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I(i + m, j + n) \cdot k(m, n) \quad (2)$$

Unde:

- I reprezintă imaginea de intrare,
- K reprezintă kernel-ul,
- $S(i, j)$ reprezintă valoarea calculată în poziția (i, j) din feature map

Această formulă arată că fiecare valoare de ieșire este obținută prin operații de înmulțire și acumulare asupra unei regiuni locale din imagine. Într-un strat de convoluție se pot folosi mai multe filtre. Fiecare filtru generează propriul feature map și poate evidenția un anumit tip de caracteristică. În primele straturi, filtrele pot identifica elemente simple, cum ar fi linii sau muchii, iar în straturile următoare pot fi extrase forme mai complexe, utile pentru clasificarea imaginii [2].

Dimensiunea rezultatului depinde de mai mulți parametri. Unul dintre aceștia este stride-ul, care reprezintă pasul cu care kernel-ul se deplasează peste imagine. Dacă stride-ul este 1, filtrul se deplasează pixel cu pixel. Dacă stride-ul este mai mare, dimensiunea feature map-ului rezultat scade. Un alt parametru este padding-ul, care presupune adăugarea unor valori, de obicei zero, la marginile imaginii pentru a controla dimensiunea ieșirii.

În cazul acestui proiect, straturile de convoluție sunt utilizate pentru prelucrarea imaginilor grayscale de 28x28 pixeli. Aceste straturi ajută rețeaua să extragă forme specifice cifrelor scrise de mână, cum ar fi linii verticale, curbe sau zone închise. Aceste caracteristici sunt apoi transmise către straturile următoare ale rețelei, unde sunt procesate și folosite pentru clasificarea cifrei.

Din punct de vedere hardware, stratul de convoluție este important deoarece conține multe operații repetitive de tip înmulțire-acumulare, cunoscute și ca operații MAC (Multiply-Accumulate). Aceste operații se repetă pentru fiecare poziție a kernelului și pentru fiecare filtru utilizat. Din acest motiv, implementarea pe FPGA poate fi avantajoasă, deoarece permite executarea mai multor operații MAC în paralel și poate reduce timpul de inferență [3].

2.3.2 Funcția ReLU

Funcția ReLU, denumită complet Rectified Linear Unit, este una dintre cele mai utilizate funcții de activare în rețelele neuronale convoluționale. Aceasta este aplicată de obicei după un strat de convoluție și are rolul de a introduce non-liniaritate în rețea [2]. Prin utilizarea funcției ReLU, rețeaua poate învăța relații mai complexe între datele de intrare și rezultatul dorit.

Relația matematică a funcției ReLU este:

$$ReLU(z) = \max(0, z) \quad (3)$$

Conform acestei relații, dacă valoarea de intrare este negativă, ieșirea funcției este 0, iar dacă valoarea de intrare este pozitivă, ieșirea este egală cu valoarea respectivă. Astfel, funcția ReLU elimină valorile negative și păstrează valorile pozitive.

Graficul funcției ReLU este prezentat în Figura 2.3.

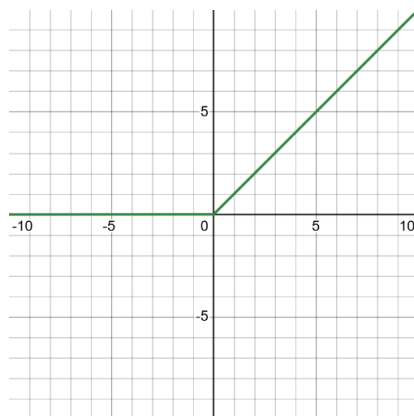


Figura 2.3. Graficul funcției ReLU.

Figura 2.3 evidențiază comportamentul funcției ReLU, pentru valori negative ieșirea este 0, iar pentru valori pozitive ieșirea crește liniar. Din punct de vedere hardware, funcția ReLU este eficientă deoarece poate fi implementată printr-o simplă comparație cu valoarea 0. În cadrul proiectului, funcția ReLU este utilizată după straturile de convoluție, pentru păstrarea activărilor relevante în procesul de clasificare a cifrelor scrise de mână.

Exemplu de aplicare ReLU asupra unui feature map este prezentat în Figura 2.4.

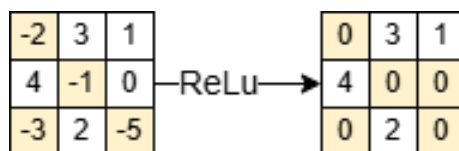


Figura 2.4. Exemplu de aplicare a funcției ReLU asupra unei hărți de caracteristici.

Figura 2.4 arată modul în care funcția ReLU este aplicată element cu element asupra unei hărți de caracteristici. Valorile negative sunt înlocuite cu 0, iar valorile pozitive sunt păstrate. În cadrul proiectului, funcția ReLU este utilizată după straturile de convoluție, pentru păstrarea activărilor relevante în procesul de clasificare a cifrelor scrise de mână.

2.3.3 Max-pooling

Max-pooling este o operație utilizată frecvent în rețelele neuronale convoluționale pentru reducerea dimensiunii datelor [2]. Aceasta se aplică de obicei după unul sau mai multe

straturi de convoluție și are rolul de a păstra informațiile cele mai importante, reducând în același timp numărul de valori care trebuie procesate în straturile următoare.

Operația de max-pooling se realizează prin împărțirea feature map-ului în ferestre de dimensiuni reduse, de exemplu 2x2. Pentru fiecare fereastră, se selectează valoarea maximă, iar aceasta este transmisă mai departe. Astfel, o regiune de 2x2 valori este înlocuită cu o singură valoare, ceea ce reduce dimensiunea datelor intermediare.

Un exemplu de aplicare a operației de max-pooling pe ferestre 2x2 este prezentat în Figura 2.5.

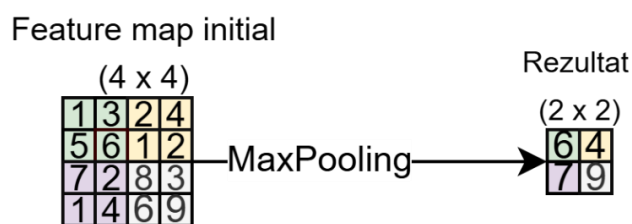


Figura 2.5. Exemplu de operație max-pooling 2x2.

Figura 2.5 arată modul în care fiecare fereastră 2x2 este redusă la valoarea maximă.

Relația generală pentru o operație de max-pooling poate fi exprimată astfel:

$$P(i, j) = \max R(i, j) \quad (4)$$

unde:

- $R(i, j)$ reprezintă regiunea locală analizată din feature map;
- $P(i, j)$ reprezintă valoarea rezultată după aplicarea operației de max-pooling.

De exemplu, pentru o fereastră de 2x2, dacă valorile sunt 1, 3, 2 și 5, rezultatul operației de max-pooling va fi 5. În acest fel, sunt păstrate cele mai puternice activări, care pot corespunde unor caracteristici importante identificate de filtrele convoluționale.

În cazul proiectului, operația de max-pooling este utilă deoarece reduce dimensiunea datelor intermediare și scade numărul de calcule necesare în straturile următoare. Acest lucru este important atât pentru performanța rețelei, cât și pentru implementarea hardware pe FPGA, deoarece reduce cantitatea de date care trebuie stocată și procesată.

2.3.4 Straturi fully-connected

Straturile fully-connected, numite și straturi complet conectate, sunt utilizate de obicei în partea finală a unei rețele neuronale convoluționale. Rolul lor este de a realiza clasificarea pe baza caracteristicilor extrase anterior de straturile de convoluție și de max-pooling. Într-un strat fully-connected, fiecare neuron este conectat la toate valorile provenite din stratul anterior.

Înainte ca datele să ajungă într-un strat fully-connected, feature map-urile obținute după straturile convoluționale sunt transformate într-un vector. Acest proces se numește flattening. Astfel, informația extrasă sub formă de matrice este reorganizată într-o formă potrivită pentru clasificare.

Calculul realizat într-un strat fully-connected este asemănător cu cel al neuronului artificial prezentat anterior. Fiecare valoare de intrare este înmulțită cu o pondere, rezultatele sunt adunate, iar la final se adaugă un termen de bias. Relația matematică poate fi exprimată astfel:

$$y_k = \sum_{i=1}^{\{n\}} w_{\{k,i\}} x_i + b_k \quad (5)$$

unde

- x_i reprezintă valorile de intrare;
- $w_{k,i}$ reprezintă ponderea asociată intrării i pentru neuronul k ;
- b_k reprezintă bias-ul neuronului k ;
- y_k reprezintă ieșirea calculată pentru neuronul k .

În cazul unei probleme de clasificare, ultimul strat fully-connected are un număr de ieșiri egal cu numărul de clase. Pentru recunoașterea cifrelor scrise de mână, rețeaua are 10 ieșiri, corespunzătoare cifrelor de la 0 la 9. Fiecare ieșire reprezintă un scor asociat unei cifre, iar cifra recunoscută este cea care are scorul cel mai mare.

În cadrul proiectului, straturile fully-connected sunt importante deoarece realizează etapa finală de decizie. După ce straturile convoluționale au extras caracteristicile relevante din imagine, stratul fully-connected folosește aceste informații pentru a determina cifra reprezentată în imaginea de intrare.

2.4 IMPLEMENTAREA REȚELELOR NEURONALE PE FPGA

Rețelele neuronale pot fi implementate pe mai multe tipuri de platforme, precum CPU, GPU sau FPGA. Alegerea platformei influențează performanța, consumul de energie, flexibilitatea și complexitatea implementării. În cazul rețelelor neuronale convoluționale, operațiile cele mai frecvente sunt operațiile de tip MAC (Multiply-Accumulate), întâlnite în straturile de convoluție și în straturile complet conectate. Deoarece aceste operații sunt repetitive, execuția lor secvențială poate conduce la timpi de procesare mai mari [3].

FPGA-urile reprezintă o soluție potrivită pentru aplicațiile embedded, deoarece permit implementarea directă în hardware a unor structuri dedicate și exploatarea paralelismului. Pentru rețelele CNN, acest aspect este important deoarece operațiile de convoluție și cele din straturile fully-connected pot fi organizate astfel încât mai multe calcule să fie executate în paralel [3].

Implementarea pe FPGA este însă mai complexă decât rularea software pe CPU sau GPU, deoarece algoritmul trebuie adaptat la resursele disponibile și la modul de execuție hardware. Pentru simplificarea dezvoltării pot fi utilizate instrumente de tip HLS, care permit descrierea algoritmului în C/C++ și generarea unei implementări hardware compatibile cu FPGA [5].

În contextul acestei lucrări, FPGA-ul este utilizat pentru accelerarea etapei de inferență a rețelei neuronale. Procesorul are rolul de a controla transferul datelor și execuția sistemului, iar logica programabilă este utilizată pentru operațiile intensive din punct de vedere computațional.

2.5 PLATFORMA ZYNQ ȘI TRANSFERUL DATELOR PRIN AXI/DMA

Platforma Zynq reprezintă o arhitectură de tip SoC (System-on-Chip), care combină într-un singur circuit un sistem de procesare și o zonă de logică programabilă. Această organizare permite realizarea unor sisteme embedded în care partea software controlează aplicația, iar partea hardware execută operații dedicate.

În cadrul familiei Zynq-7000, arhitectura este împărțită în două componente principale: Processing System (PS) și Programmable Logic (PL). Partea PS conține procesorul ARM, controlerele de memorie și perifericele integrate, iar partea PL reprezintă zona FPGA în care pot fi implementate circuite digitale personalizate [6]. În cadrul sistemului realizat, PS-ul rulează aplicația de control și gestionează comunicația, iar PL-ul conține acceleratorul CNN.

Comunicarea dintre PS și PL se realizează prin interfețe AXI. AXI-Lite este utilizată pentru accesarea registrelor de control ale acceleratorului, iar AXI-Stream este utilizată pentru transferul continuu al datelor către și dinspre accelerator [7].

Pentru transferuri eficiente între memoria DDR și accelerator este utilizat mecanismul DMA (Direct Memory Access). DMA permite mutarea datelor fără ca procesorul să copieze manual fiecare valoare [8]. În acest mod, procesorul pregătește datele și inițiază transferul, iar DMA transmite imaginea către accelerator și preia rezultatele obținute.

2.6 REPREZENTAREA NUMERICĂ ÎN HARDWARE

Reprezentarea numerică a datelor este un aspect important în implementarea rețelelor neuronale pe hardware. În etapa de antrenare, modelele sunt de obicei dezvoltate folosind

numere în virgulă mobilă, deoarece acestea oferă precizie ridicată și sunt ușor de utilizat în mediile software. Totuși, în implementările hardware, operațiile în virgulă mobilă pot consuma mai multe resurse și pot crește complexitatea circuitului.

Pentru a reduce complexitatea implementării hardware, se pot utiliza reprezentări numerice de tip fixed-point. În acest caz, valorile sunt reprezentate folosind un număr fix de biți, împărțiți între partea întreagă și partea fracționară. Spre deosebire de reprezentarea în virgulă mobilă, poziția virgulei este fixă, ceea ce simplifică operațiile aritmetice. Acest aspect este important în cazul FPGA-urilor, unde numărul de resurse logice, blocuri DSP și memorie este limitat.

Totuși, reprezentarea fixed-point introduce și limitări. Dacă numărul total de biți este prea mic, pot apărea erori de cuantizare sau depășiri ale intervalului de reprezentare. Din acest motiv, alegerea formatului numeric trebuie să realizeze un compromis între acuratețea modelului și eficiența implementării hardware.

În HLS pot fi utilizate tipuri de date fixed-point, precum `ap_fixed<W,I>`, unde `W` reprezintă numărul total de biți, iar `I` reprezintă numărul de biți utilizați pentru partea întreagă. Restul biților sunt utilizați pentru partea fracționară. De exemplu, tipul `ap_fixed<16,6>` folosește 16 biți în total, dintre care 6 biți pentru partea întreagă și 10 biți pentru partea fracționară [5].

În contextul implementării unei rețele neuronale pe FPGA, reprezentarea numerică influențează atât precizia inferenței, cât și resursele hardware necesare. De aceea, trecerea de la valori în virgulă mobilă la valori fixed-point trebuie analizată și testată în raport cu performanța modelului.

2.7 SISTEMUL DE OPERARE EMBEDDED PETALINUX

PetaLinux este un set de instrumente oferit de AMD pentru dezvoltarea sistemelor embedded Linux pe platforme bazate pe FPGA și SoC. Acesta permite generarea, configurarea și construirea unei imagini Linux adaptate pentru hardware-ul proiectat în Vivado, incluzând componente precum bootloader, kernel Linux, device tree și root file system [9]. Într-un sistem bazat pe Zynq, PetaLinux poate fi utilizat pentru rularea aplicațiilor software pe Processing System și pentru integrarea perifericelor hardware implementate în Programmable Logic.

3 ARHITECTURA SISTEMULUI PROPOS

Acest capitol prezintă arhitectura generală a sistemului realizat pentru recunoașterea cifrelor scrise de mână. Sunt descrise componentele principale ale sistemului, împărțirea sarcinilor între partea software și partea hardware, precum și traseul datelor de la imaginea capturată până la rezultatul afișat utilizatorului.

3.1 FUNCȚIONALITĂȚILE SISTEMULUI PROPOS

Sistemul propus are rolul de a recunoaște cifre scrise de mână, corespunzătoare claselor de la 0 la 9. Utilizatorul poziționează una sau mai multe cifre în fața camerei, iar sistemul preia imaginea, pregătește datele de intrare, execută inferența CNN pe acceleratorul hardware implementat pe FPGA și afișează rezultatul obținut.

Pentru achiziția imaginii este utilizată camera telefonului. Conectarea acesteia la laptop este realizată prin aplicația externă Camo, care permite folosirea telefonului ca webcam. Aplicația dezvoltată în cadrul proiectului nu comunică direct cu telefonul, ci preia fluxul video disponibil pe laptop prin OpenCV.

Funcționalitățile principale ale sistemului sunt:

- capturarea imaginii de la camera telefonului utilizată ca webcam;
- extragerea regiunii de interes din imaginea capturată;
- preprocesarea imaginii pentru separarea cifrei de fundal;
- detectarea și separarea cifrelor din imaginea binară;
- normalizarea fiecărei cifre la dimensiunea de 28×28 pixeli;
- transmiterea fiecărei cifre către placa ZedBoard sub forma unui vector de 784 bytes;
- clasificarea cifrei cu ajutorul acceleratorului CNN implementat în FPGA;
- returnarea rezultatului către aplicația PC și afișarea cifrei recunoscute.

În cazul în care în regiunea de interes sunt detectate mai multe cifre, acestea sunt procesate separat. Aplicația le ordonează de la stânga la dreapta, transmite fiecare cifră către ZedBoard și concatenează rezultatele primite. Astfel, sistemul poate afișa nu doar o singură cifră, ci și un număr format din mai multe cifre detectate în aceeași imagine.

3.2 ARHITECTURA HARDWARE-SOFTWARE

Sistemul propus este organizat sub forma unei arhitecturi hardware-software, în care aplicația de pe PC realizează achiziția și pregătirea datelor, iar placa ZedBoard execută inferența CNN cu ajutorul acceleratorului hardware. Pentru o prezentare clară, arhitectura este descrisă pe două niveluri: arhitectura de ansamblu a sistemului și organizarea hardware-software a platformei embedded ZedBoard.

Arhitectura de ansamblu a sistemului propus este prezentată în Figura 3.1.

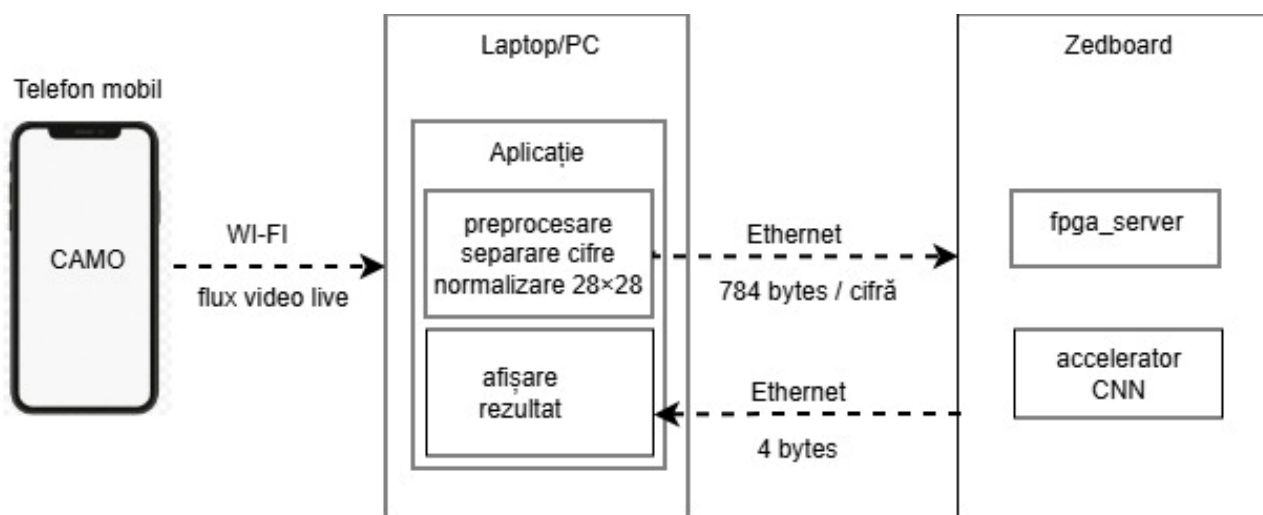


Figura 3.1. Arhitectura de ansamblu a sistemului propus.

Figura 3.1 prezintă traseul general al datelor în sistem, evidențiind comunicarea dintre telefon, laptop și placa ZedBoard. Telefonul mobil este utilizat ca sursă video prin aplicația externă Camo, instalată pe telefon și pe laptop, care permite folosirea camerei telefonului ca webcam pentru laptop. Fluxul video este transmis prin Wi-Fi, iar aplicația dezvoltată în cadrul proiectului nu comunică direct cu telefonul, ci preia imaginea disponibilă pe laptop prin OpenCV.

Pe laptop rulează aplicația Python, care realizează preprocesarea, separarea cifrelor și normalizarea acestora la dimensiunea de 28×28 pixeli. Fiecare cifră normalizată este transmisă către placa ZedBoard prin Ethernet/TCP-IP, sub forma unui vector de 784 bytes. După procesarea pe ZedBoard, rezultatul este returnat către aplicația de pe PC prin aceeași conexiune Ethernet/TCP-IP, sub forma unui răspuns de 4 bytes și este afișat utilizatorului.

Pentru detalierea părții embedded a sistemului și a organizării interne a platformei ZedBoard, este utilizată reprezentarea din Figura 3.2. Această schemă nu mai include fluxul video și aplicația PC în detaliu, deoarece acestea au fost prezentate anterior la nivel de ansamblu. Accentul este pus pe componentele interne ale plăcii ZedBoard și pe separarea dintre partea software și partea hardware.

În Processing System rulează procesorul ARM Cortex-A9, sistemul de operare PetaLinux și aplicația fpga_server. PetaLinux oferă mediul de execuție pentru aplicația embedded, iar fpga_server gestionează comunicația cu aplicația PC și controlul transferurilor către partea hardware.

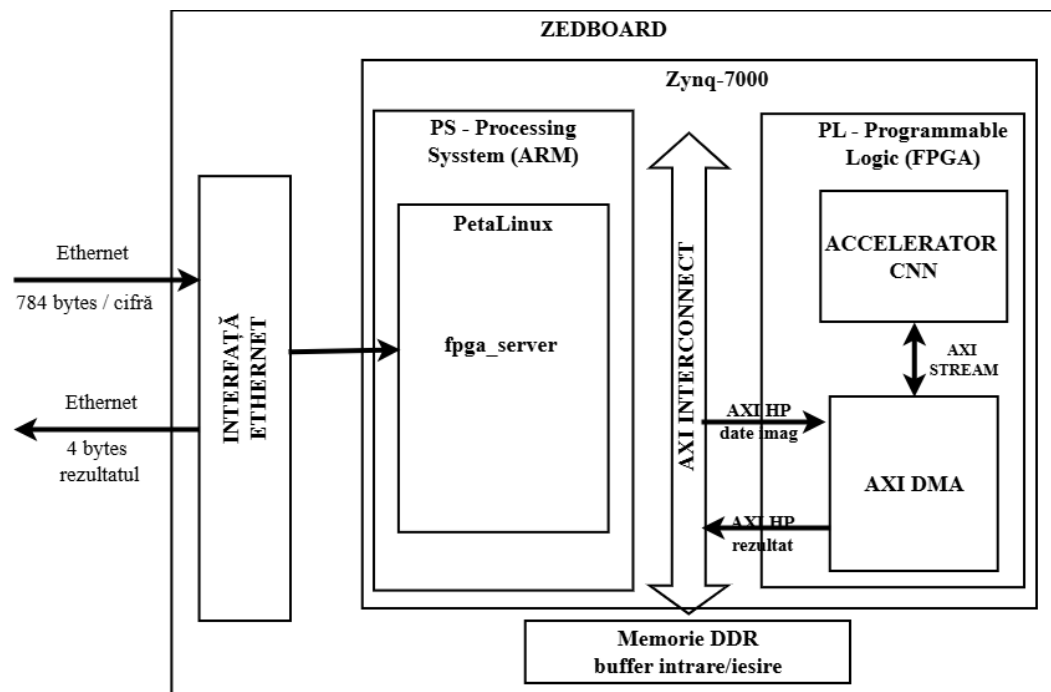


Figura 3.2. Arhitectura hardware-software a sistemului embedded de pe ZedBoard.

Figura 3.2 prezintă organizarea hardware-software a sistemului embedded de pe placa ZedBoard. Memoria DDR este utilizată ca zonă de stocare pentru bufferul de intrare și pentru rezultatul obținut după procesare. În Programmable Logic sunt implementate modulul AXI DMA și acceleratorul CNN. AXI DMA realizează transferul datelor între memoria DDR și accelerator, folosind porturile AXI HP prin intermediul AXI Interconnect. Comunicația dintre AXI DMA și acceleratorul CNN se realizează prin interfața AXI Stream. Prin această organizare, Processing System-ul are rol de comunicație și control, iar Programmable Logic-ul execută partea de calcul intensiv a sistemului. Fluxul complet al datelor, de la imaginea capturată până la rezultatul afișat, este prezentat în secțiunea următoare.

3.3 FLUXUL DE DATE ÎN SISTEM

Fluxul de date al sistemului pornește de la imaginea capturată cu ajutorul camerei telefonului mobil și se încheie cu afișarea cifrei recunoscute în aplicația de pe PC. Telefonul este utilizat ca webcam prin aplicația Camo, iar fluxul video ajunge pe laptop, unde este preluat de aplicația Python prin OpenCV.0

Pentru o prezentare mai clară a etapelor parcurse de date, fluxul sistemului este sintetizat în Figura 3.3.

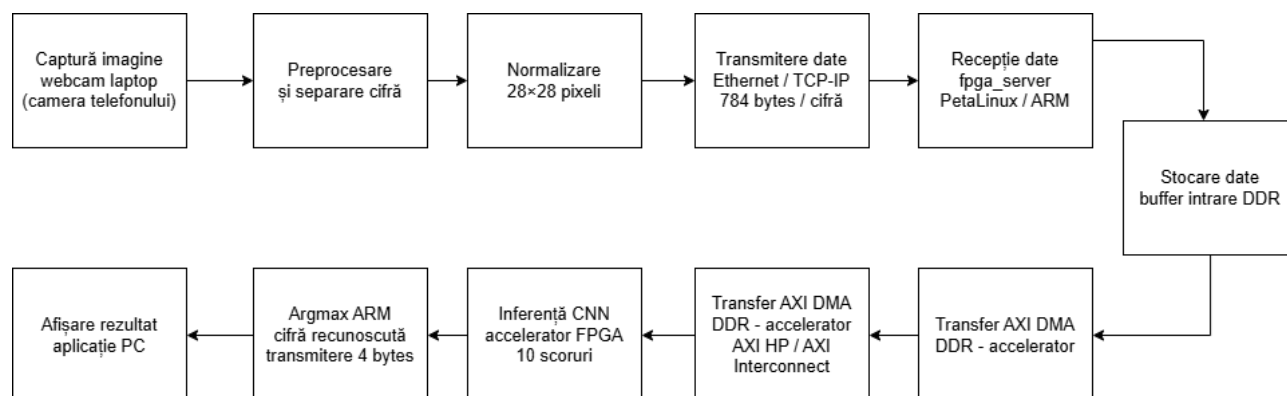


Figura 3.3. Fluxul de date pentru recunoașterea unei cifre.

Figura 3.3 prezintă succesiunea principalelor etape prin care trece informația în sistem. Imaginea este preluată de aplicația PC, unde este extrasă regiunea de interes și sunt aplicate etapele de preprocesare, separare și normalizare. Fiecare cifră detectată este adusă la dimensiunea de 28x28 pixeli și este transformată într-un vector de 784 valori.

Valorile transmise către ZedBoard sunt codificate în format raw fixed-point. Pixelii de fundal sunt reprezentați prin valoarea 0, iar pixelii activi ai cifrei prin valoarea 16, care corespunde valorii 1.0 în formatul `ap_fixed<8,4>`. Pentru fiecare cifră, aplicația PC transmite către ZedBoard un bloc de 784 bytes prin conexiune Ethernet/TCP-IP.

Pe placa ZedBoard, datele sunt recepționate de aplicația `fpga_server`, care rulează în PetaLinux pe procesorul ARM. Datele primite sunt pregătite în memoria DDR și sunt transferate către acceleratorul CNN prin AXI DMA. Transferul dintre memoria DDR și AXI DMA se realizează prin porturile AXI HP, prin intermediul AXI Interconnect, iar datele ajung la accelerator prin interfața AXI Stream. Acceleratorul CNN execută inferența și generează scorurile corespunzătoare claselor 0–9. Aceste scoruri sunt returnate către partea software de pe ZedBoard, unde aplicația `fpga_server` determină cifra recunoscută prin alegerea scorului maxim. Către aplicația PC este transmis doar rezultatul final, sub forma unui răspuns de 4 bytes. Dacă în regiunea de interes sunt detectate mai multe cifre, acestea sunt procesate pe rând. Aplicația PC le ordonează de la stânga la dreapta, transmite separat fiecare cifră către ZedBoard și afișează rezultatele primite în aceeași ordine.

4 IMPLEMENTAREA APLICAȚIEI SOFTWARE

Acest capitol prezintă implementarea aplicației software dezvoltate pe PC. Aplicația este realizată în Python și are rolul de a prelua imaginea de la cameră, de a extrage cifrele scrise de mână, de a le transforma în formatul necesar acceleratorului CNN și de a transmite datele către placa ZedBoard.

Pentru implementare au fost utilizate bibliotecile OpenCV, NumPy, socket și os. OpenCV este folosită pentru capturarea imaginilor, prelucrarea acestora și afișarea interfeței, NumPy pentru operații pe matrici de pixeli, socket pentru comunicația TCP/IP, iar os pentru salvarea datelor de test.

4.1 INTERFAȚA APLICAȚIEI

Interfața aplicației software a fost realizată în Python, utilizând biblioteca OpenCV. Aceasta permite urmărirea în timp real a imaginii capturate, a regiunii de interes, a cifrelor detectate și a rezultatului primit de la placa ZedBoard.

Interfața aplicației este prezentată în Figura 4.1.



Figura 4.1. Interfața aplicației software în timpul rulării.

Figura 4.1 prezintă fereastra principală utilizată pentru demonstrarea funcționării sistemului. În partea superioară a ferestrei sunt afișate imaginea capturată de cameră și regiunea de interes în care sunt detectate cifrele. În partea inferioară sunt afișate panoul de control, starea aplicației, rezultatul transmis de acceleratorul FPGA și imaginile de 28×28 pixeli obținute după normalizare. Această organizare permite verificarea rapidă a întregului flux de lucru: achiziția imaginii, detecția cifrelor, pregătirea datelor de intrare și afișarea predicției obținute hardware.

Rolul panourilor din interfață este prezentat în Tabelul 4.1.

Tabel 4.1. Rolul panourilor din interfața aplicației software.

Panou	Rol
Camera live	Afișează imaginea capturată în timp real și marchează regiunea de interes în care trebuie poziționate cifrele.
ROI cu detecții și predicții	Afișează zona analizată de aplicație și evidențiază componentele detectate corespunzătoare cifrelor.
Control și rezultat FPGA	Afișează starea calibrării, numărul de cifre detectate, valorile pragurilor, comenzile disponibile și rezultatul primit de la FPGA.
Cifre 28×28 normalizate	Afișează cifrele extrase și normalizate în formatul utilizat la intrarea rețelei CNN.

În panoul Camera live sunt desenate două chenare. Chenarul mare reprezintă regiunea de interes, adică zona extrasă din cadrul video și trimisă către etapa de preprocesare. În implementare, această zonă are dimensiunea maximă de 420×420 pixeli și este poziționată central în imagine. Chenarul interior are rol orientativ și indică zona în care este recomandat să fie poziționată cifra, astfel încât aceasta să nu fie prea aproape de marginea regiunii de interes.

Aplicația este controlată prin intermediul următoarelor taste:

- B – calibrarea fundalului
- C – trimite cifrele detectate către FPGA
- D – fereastra debug
- S – salvează datele de test
- Q/ESC – închide aplicația

În partea superioară a ferestrei sunt disponibile trei reglaje:

- diff_thr – stabilește pragul diferenței față de fundalul calibrat
- min_area - elimină componentele prea mici
- max_area_% - elimină componentele prea mari

Pentru verificarea etapelor intermediare de preprocesare, aplicația include o fereastră separată de debug. Această fereastră este utilă în timpul testării, deoarece permite observarea imaginilor rezultate după conversia în tonuri de gri, binarizare și curățarea zgomotului.

Fereastra de debug este prezentată în Figura 4.2.

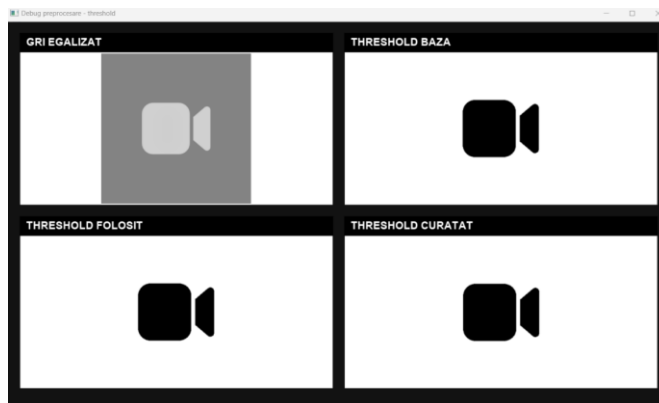


Figura 4.2. Fereastra de debug

Figura 4.2 prezintă imaginile intermediare folosite pentru verificarea detecției cifrelor. Sunt afișate imaginea în tonuri de gri egalizată, binarizarea de bază, imaginea binară selectată pentru detecție și imaginea finală obținută după operațiile de curățare. Prin această fereastră pot fi observate mai ușor efectele pragurilor și ale operațiilor morfologice asupra imaginii analizate.

Rolul imaginilor din fereastra de debug este prezentat în Tabelul 4.2.

Tabelul 4.2. Rolul imaginilor afișate în fereastra de debug.

Imagine debug	Rol
Gri egalizat	Afișează regiunea de interes după conversia în tonuri de gri și egalizarea contrastului prin CLAHE.
Threshold bază	Afișează rezultatul binarizării inițiale.
Threshold folosit	Afișează imaginea binară selectată pentru detecție.
Threshold curățat	Afișează imaginea finală după operațiile morfologice de reducere a zgomotului.

Prin această interfață, utilizatorul poate urmări atât funcționarea normală a sistemului, cât și etapele interne de preprocesare necesare pentru validarea detecției. Separarea ferestrei principale de fereastra de debug face ca aplicația să fie mai clară în timpul demonstrației, păstrând în același timp posibilitatea de analiză detaliată atunci când este necesară ajustarea parametrilor.

4.2 PREPROCESAREA IMAGINII

În aplicație, cifra este căutată doar în zona încadrată afișată pe ecran. Utilizatorul poziționează foaia astfel încât cifra sau cifrele să se afle în acest chenar. În cod, această zonă este extrasă din cadrul video și are dimensiunea maximă 420×420 pixeli.

Preprocesarea are rolul de a transforma regiunea de interes într-o imagine binară, în care cifra este reprezentată prin pixeli albi, iar fundalul prin pixeli negri. Această reprezentare simplifică etapa de detecție a componentelor care pot reprezenta cifre.

Ordinea preprocesării este prezentată în Tabelul 4.3.

Tabelul 4.3. Ordinea etapelor de preprocesare a imaginii

Pas	Operație	Rezultat
1	Extragerea zonei de lucru	Se obține regiunea de interes din cadrul video.
2	Conversia în tonuri de gri	Se elimină informația de culoare, iar fiecare pixel este reprezentat printr-o singură valoare de intensitate.
3	Aplicarea CLAHE	Se îmbunătățește contrastul local dintre cifră și fundal.
4	Filtrare Gaussiană	Se reduc variațiile mici și punctele de zgomot.
5	Decuparea marginilor interioare	Se elimină o margine a ROI-ului pentru a evita detecțiile false la marginea zonei de lucru.
6	Threshold Otsu și threshold adaptiv	Se obțin două variante de imagini alb-negru.
7	Alegerea threshold-ului de bază	Se selectează varianta mai potrivită pentru cadrul curent.
8	Compararea cu fundalul calibrat	Dacă există calibrare, se obține o imagine binară pe baza diferenței față de fundal.
9	Alegerea threshold-ului folosit	Se alege imaginea binară utilizată efectiv pentru detecție.
10	Operații morfologice	Se elimină zgomotul rămas și se obține imaginea finală pentru detecția componentelor.

Conversia în tonuri de gri reduce imaginea la un singur canal de intensitate, ceea ce simplifică prelucrarea ulterioară. Pentru îmbunătățirea contrastului este aplicată metoda CLAHE, iar filtrarea Gaussiană este utilizată pentru reducerea zgomotului de nivel mic. După această etapă, aplicația taie o margine interioară a zonei analizate, pentru a evita influența marginilor ROI-ului asupra detecției.

Pentru obținerea imaginii binare sunt utilizate două metode: threshold Otsu și threshold adaptiv. Ambele metode sunt aplicate pe imaginea curentă obținută după conversia în gri, CLAHE, filtrare Gaussiană și decuparea marginilor interioare. Fiecare metodă produce o imagine alb-negru, iar aplicația selectează varianta mai potrivită prin compararea numărului de pixeli albi. Imaginea aleasă este denumită threshold de bază.

Calibrarea fundalului este utilizată pentru a îmbunătăți separarea cifrei în situații cu umbre, iluminare neuniformă sau variații ale fundalului. La apăsarea tastei B, aplicația salvează fundalul după conversia în gri, aplicarea CLAHE, filtrarea Gaussiană și decuparea marginilor interioare. Imaginea salvată nu este binară, ci reprezintă fundalul prelucrat până la etapa de crop.

Când apare o cifră în zona de lucru, imaginea curentă este prelucrată în același mod, până la aceeași etapă de crop. Aplicația compară fundalul memorat cu imaginea curentă, iar diferența obținută este binarizată folosind parametrul DIFF_THR. Dacă imaginea rezultată conține un număr rezonabil de pixeli albi, aceasta este folosită pentru detecție. În caz contrar, aplicația folosește threshold-ul de bază obținut prin Otsu sau threshold adaptiv.

După alegerea threshold-ului folosit, sunt aplicate operații morfologice de curățare. Acestea elimină punctele izolate și reduc micile întreruperi ale formei cifrei, așa cum este prezentat în următoarea în Figura 4.3 .

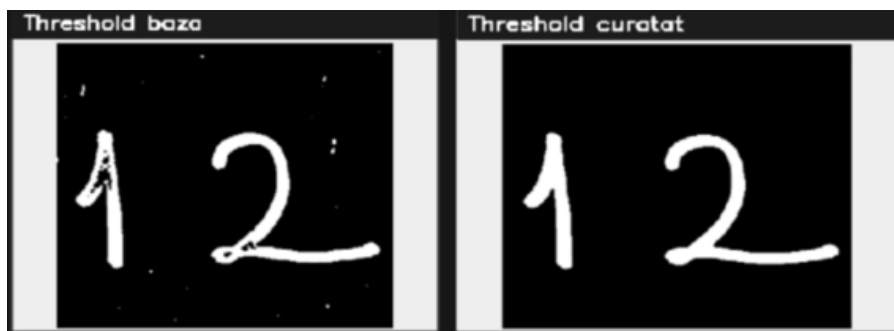


Figura 4.3. Comparație între imaginea binară inițială și imaginea binară curățată.

În Figura 4.3 se observă diferența dintre imaginea Threshold bază și imaginea Threshold curățat. În prima imagine apar puncte albe de zgomot, iar după operațiile de curățare rămâne în principal forma cifrelor.

4.3 SEPARAREA ȘI NORMALIZAREA CIFRELOR

După etapa de preprocesare, aplicația obține imaginea Threshold curățat, în care cifra este reprezentată prin pixeli albi, iar fundalul prin pixeli negri. Această imagine este utilizată pentru separarea zonelor care pot reprezenta cifre.

Aplicația caută regiuni albe continue în imaginea binară. O astfel de regiune se numește componentă conectată și reprezintă un grup de pixeli albi aflați în contact unii cu alții. Într-o situație ideală, fiecare cifră scrisă formează o componentă separată. În practică însă, pot apărea și componente false, generate de puncte de zgomot, umbre, reflexii sau urme de pe hârtie. De aceea, fiecare regiune este verificată înainte de a fi acceptată.

Criteriile de filtrare sunt prezentate în Tabelul 4.4.

Tabel 4.4. Criterii de filtrare a zonelor detectate.

Criteriu	Rol în aplicație
Arie	Elimină punctele mici și regiunile prea mari.
Lățime și înălțime	Elimină regiunile cu dimensiuni nepotrivite.
Raport înălțime/lățime	Elimină regiunile cu proporții greșite.
Poziție față de margine	Elimină regiunile tăiate de marginea zonei de lucru.
Număr de pixeli activi	Elimină regiunile aproape goale după normalizare.

Prin aplicarea acestor criterii, aplicația păstrează doar regiunile care au dimensiuni și proporții apropiate de cele ale unei cifre. Regiunile prea mici sunt considerate zgomot, iar cele prea mari pot proveni din pete, umbre sau zone unite incorect. De asemenea, componentele aflate prea aproape de marginea zonei de lucru sunt eliminate, deoarece pot reprezenta cifre tăiate parțial.

Zonele acceptate sunt marcate cu chenare în panoul „ROI cu detecții și predicții”. Dacă sunt detectate mai multe cifre, acestea sunt sortate de la stânga la dreapta, pentru a păstra ordinea numărului scris.

După separare, fiecare cifră acceptată este normalizată înainte de a fi trimisă către rețeaua CNN. Normalizarea este necesară deoarece acceleratorul primește imagini cu dimensiunea fixă de 28×28 pixeli. În plus, cifra trebuie să fie centrată și să păstreze o margine neagră în jur, asemănător imaginilor din setul MNIST.

Normalizarea unei cifre este prezentată în Tabelul 4.5.

Tabel 4.5. Normalizarea unei cifre detectate.

Pas	Operație
1	se extrage zona în care se află cifra
2	cifra încadrată într-o imagine pătrată
3	imaginea este redimensionată la 18×18 pixeli
4	cifra este plasată în centrul unei imagini 28×28
5	cifra este recentrată folosind centrul de masă

Încadrarea cifrei într-o imagine pătrată este necesară pentru a păstra proporțiile acesteia la redimensionare. Dacă cifra ar fi redimensionată direct dintr-un dreptunghi la 28×28 pixeli, forma ei s-ar putea deforma. De aceea, aplicația creează mai întâi o imagine pătrată, apoi redimensionează cifra la 18×18 pixeli și o așază în centrul unei imagini negre de 28×28 pixeli. Recentrarea folosind centrul de masă are rolul de a poziționa cifra cât mai aproape de centrul

imaginii finale. Centrul de masă este calculat pe baza distribuției pixelilor albi, iar imaginea este deplasată astfel încât cifra să fie centrată mai bine. Rezultatul final este o imagine de 28×28 pixeli, compatibilă cu intrarea acceleratorului CNN.

Etapă de separare și normalizare este prezentată în Figura 4.4.

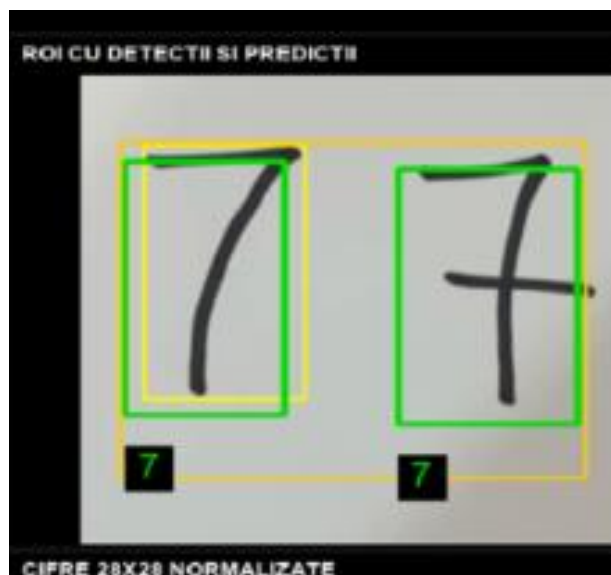


Figura 4.4. Separarea și normalizarea cifrelor.

Figura 4.4 prezintă cifrele detectate și forma finală 28×28 transmisă către ZedBoard.

4.4 TRIMITEREA CIFRELOR CĂTRE ZEDBOARD

După separarea și normalizarea cifrelor, aplicația transmite fiecare imagine de 28×28 pixeli către placa ZedBoard. Comunicarea este realizată prin TCP/IP, folosind adresa IP 192.168.1.10 și portul 12345.

În aplicație, trimiterea unei cifre este realizată prin funcția `send_to_board()`. Aceasta primește imaginea normalizată, o transformă într-un vector liniar și o transmite către ZedBoard. Imaginea de intrare are dimensiunea 28×28 pixeli, deci rezultă un vector de 784 valori.

Înainte de transmitere, imaginea este convertită într-un format numeric simplu: pixelii de fundal sunt codificați cu valoarea 0, iar pixelii activi ai cifrei sunt codificați cu valoarea 16. Valoarea 16 este utilizată deoarece reprezintă valoarea 1.0 în formatul fixed-point folosit la intrarea acceleratorului. Detaliile privind reprezentarea fixed-point sunt prezentate în capitolul dedicat implementării hardware.

Această etapă face legătura dintre aplicația software de pe PC și acceleratorul implementat pe FPGA. Pe partea de PC nu se mai realizează clasificarea propriu-zisă, ci doar pregătirea datelor într-un format compatibil cu intrarea acceleratorului. Clasificarea cifrei este

realizată pe placa ZedBoard, iar aplicația PC are rolul de a trimite datele și de a afișa rezultatul primit.

Formatul datelor transmise prin TCP/IP este prezentat în Tabelul 4.6.

Tabelul 4.6. Formatul datelor transmise prin TCP/IP

Date	Dimensiune	Rol
Imaginea unei cifre	784 bytes	Date trimise către ZedBoard.
Răspunsul primit	4 bytes	Cifra recunoscută.

Aplicația deschide o conexiune TCP către placa ZedBoard, transmite cei 784 bytes corespunzători imaginii și apoi așteaptă un răspuns de 4 bytes. Răspunsul este interpretat ca un număr întreg și reprezintă cifra recunoscută.

Dacă sunt detectate mai multe cifre, acestea nu sunt trimise într-un singur bloc. Aplicația le procesează pe rând, în ordinea poziției lor în imagine, de la stânga la dreapta. Pentru fiecare cifră se realizează o transmitere separată, iar rezultatele primite sunt concatenate și afișate în interfața aplicației.

Această secțiune descrie doar modul în care aplicația PC pregătește și transmite datele către ZedBoard. Modul în care datele sunt preluate și procesate pe placă este prezentat în capitolul dedicat integrării sistemului embedded.

5 DEZVOLTAREA ȘI EVALUAREA MODELULUI CNN ÎN SOFTWARE

Acest capitol prezintă modelul CNN utilizat pentru recunoașterea cifrelor scrise de mână. Modelul a fost realizat și antrenat în Python, folosind TensorFlow/Keras, iar rezultatele obținute în software sunt utilizate ca referință pentru implementarea hardware.

Rețeaua primește la intrare o imagine grayscale de 28×28 pixeli și produce la ieșire 10 scoruri, corespunzătoare claselor 0–9. Clasa prezisă este determinată prin operația argmax .

5.1 SETUL DE DATE ȘI PREGĂTIREA IMAGINILOR

Pentru antrenarea modelului a fost utilizat setul de date MNIST, completat cu un set de imagini proprii. MNIST reprezintă baza principală de antrenare, deoarece conține un număr mare de exemple de cifre scrise de mână, organizate în cele 10 clase corespunzătoare cifrelor de la 0 la 9.

Pe lângă setul MNIST, a fost utilizat și un set de imagini proprii, obținute din cifre capturate cu camera și prelucrate anterior, fiind adăugat pentru ca modelul să se adapteze mai bine la imaginile generate de aplicația practică. Deoarece numărul acestor imagini este mai mic decât cel din MNIST, ele au fost repetate de trei ori în setul de antrenare.

Înainte de antrenare, toate imaginile utilizate sunt aduse la același format de intrare: grayscale, 28×28 pixeli, cu valori normalizate în intervalul $[0,1]$. În cazul imaginilor proprii, este verificată și polaritatea, astfel încât cifra să fie deschisă la culoare pe fundal închis.

Datele utilizate în procesul de antrenare și testare sunt prezentate în Tabelul 5.1.

Tabelul 5.1. Setul de date utilizat.

Set de date	Număr imagini	Utilizare
MNIST train	55.000	antrenare
MNIST validation	5.000	validare în timpul antrenării
MNIST test	10.000	evaluare finală
Imagini proprii	35.750	întărirea recunoașterii pe imagini proprii

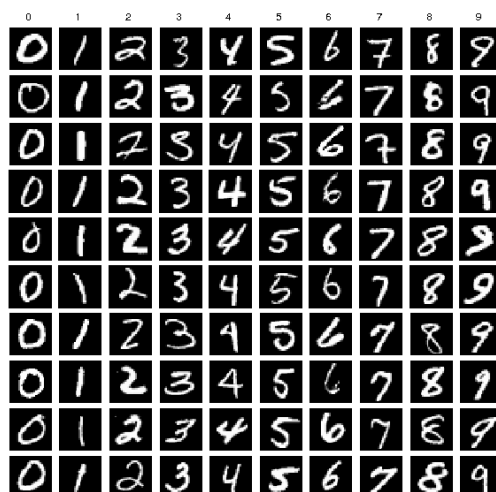


Figura 5.1. Exemple de imagini de setul din date MNIST [4].

Figura 5.1 prezintă exemple de cifre din setul MNIST, utilizat ca bază principală pentru antrenarea modelului.

Pentru creșterea robusteții modelului, imaginile proprii au fost generate în mai multe variante, prin rotații, deplasări mici și modificări ale dimensiunii cifrei. Aceste variante au rolul de a simula situații care pot apărea în utilizarea practică, atunci când cifra este ușor înclinată, poziționată diferit sau capturată la o dimensiune diferită. În plus, în timpul antrenării a fost aplicată o augmentare suplimentară redusă, formată din rotație, translație, zoom și modificare de contrast. Această augmentare este aplicată dinamic, fără salvarea unor imagini noi pe disc, și ajută modelul să devină mai robust la variațiile imaginilor de intrare.

Setul propriu conține 35.750 de imagini proprii preprocesate. În etapa de antrenare, acest set a fost repetat de trei ori, rezultând 107.250 de imagini proprii folosite efectiv la antrenare.

Tabelul 5.2. Transformările aplicate pentru augmentarea datelor.

Transformare	Valoare utilizată	Rol
RandomRotation	0,03, aproximativ $\pm 10,8^\circ$	cifre ușor înclinate
RandomTranslation	0,06	poziționare diferită în imagine
RandomZoom	0,06	variații ale dimensiunii cifrei
RandomContrast	0,10	variații de contrast/iluminare

Tabelul 5.2. Transformările aplicate pentru augmentarea datelor.



Figura 5.2. Exemple de variații utilizate pentru augmentarea datelor.

Figura 5.2 prezintă variații obținute prin augmentare, folosite pentru a expune modelul la cifre ușor rotite, deplasate sau capturate cu dimensiuni diferite.

5.2 ARHITECTURA REȚELEI NEURONALE

Pentru clasificarea imaginilor a fost utilizată o rețea neuronală convoluțională compactă. Modelul este alcătuit din două blocuri convoluționale, urmate de o etapă de vectorizare și de două straturi fully-connected.

Structura modelului este prezentată în Tabelul 5.3.

Tabelul 5.3. Arhitectura modelului CNN.

Strat	Parametri principali	Dimensiune ieșire
Input	Imagine grayscale	28×28×1
Conv1 + ReLU	4 filtre, kernel 3×3, padding valid	26×26×4
MaxPooling1	Fereastră 2×2	13×13×4
Conv2 + ReLU	8 filtre, kernel 3×3, padding valid	11×11×8
MaxPooling2	Fereastră 2×2	5×5×8
Flatten	Conversie în vector	200
FC1 + ReLU	32 neuroni	32
FC2	10 neuroni	10

Pentru o reprezentare mai clară a succesiunii straturilor și a dimensiunilor intermediare, arhitectura modelului este ilustrată în Figura 5.3.

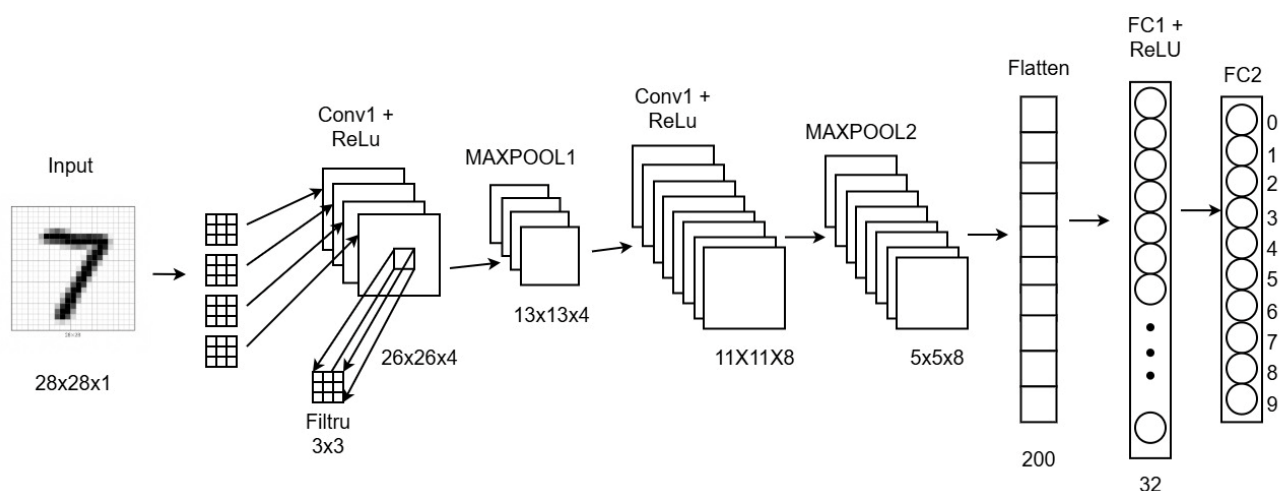


Figura 5.3. Arhitectura rețelei CNN

Primul strat convoluțional aplică 4 filtre de dimensiune 3×3 pe imaginea de intrare de dimensiune 28×28×1. Deoarece se utilizează padding valid, dimensiunea spațială se reduce la 26×26, iar cele 4 filtre generează 4 hărți de caracteristici. După prima operație de max-pooling, dimensiunea hărților se reduce de la 26×26×4 la 13×13×4.

Al doilea strat convoluțional aplică 8 filtre de dimensiune 3×3 asupra rezultatului precedent. În urma acestei operații se obțin 8 hărți de caracteristici de dimensiune 11×11, rezultând o ieșire de 11×11×8. După a doua operație de max-pooling, dimensiunea datelor devine

5×5×8. Rezultatul final al blocurilor convoluționale este transformat prin operația Flatten într-un vector de 200 de valori. Acest vector este procesat de stratul fully-connected FC1, care conține 32 de neuroni și utilizează funcția de activare ReLU. Stratul final FC2 produce cele 10 scoruri corespunzătoare claselor 0–9. Modelul returnează direct cele 10 scoruri, iar clasa prezisă este obținută prin operația argmax. Această alegere simplifică implementarea hardware, deoarece acceleratorul trebuie să calculeze doar scorurile finale, fără operații suplimentare pentru transformarea acestora în probabilități.

5.3 ANTRENAREA MODELULUI

Modelul a fost antrenat în Python, utilizând biblioteca TensorFlow/Keras. Scopul antrenării este determinarea parametrilor rețelei, adică a greutăților și bias-urilor, pe baza imaginilor din setul de date.

Pentru antrenare a fost utilizat algoritmul Adam, cu rata de învățare 0,001. Funcția de pierdere folosită este Categorical Crossentropy, cu parametrul from_logits=True, deoarece ultimul strat al rețelei produce direct scoruri pentru cele 10 clase. Pentru controlul procesului de antrenare au fost utilizate mai multe mecanisme. EarlyStopping oprește antrenarea dacă acuratețea pe setul de validare nu se mai îmbunătățește, reducând riscul de supraantrenare. ReduceLROnPlateau reduce rata de învățare atunci când funcția de pierdere pe validare nu mai scade, iar ModelCheckpoint salvează automat varianta modelului cu cea mai bună performanță pe setul de validare.

Parametrii utilizați la antrenare sunt prezentați în Tabelul 5.4.

Tabelul 5.4. Parametrii utilizați la antrenarea modelului.

Parametru	Valoare
Bibliotecă	TensorFlow/Keras
Batch size	128
Număr maxim de epoci	40
Optimizer	Adam
Rată de învățare	0.001
Funcție de pierdere	Categorical Crossentropy
Early stopping	patience = 8
Model salvat	mnist_manual_cnn_best.h5

După finalizarea antrenării, modelul a fost evaluat în mediul software. Pentru evaluare au fost urmărite două valori:

- acuratețea, care indică procentul imaginilor clasificate corect;
- loss-ul, care indică eroarea calculată de funcția de pierdere.

Rezultatele obținute pentru modelul în format Float32 sunt prezentate în Tabelul 5.5.

Tabelul 5.5. Rezultatele modelului CNN în format Float32.

Model	Set de testare	Număr imagini	Acuratețe	Loss
CNN Float32	MNIST test	10.000	98,33%	0,057165
CNN Float32	DATASET_MNIST_PROCESAT	35.750	99,92%	0,004422

Rezultatele din Tabelul 5.5 arată că modelul obține o acuratețe ridicată pe setul MNIST test, format din imagini care nu au fost utilizate în procesul de antrenare. De asemenea, rezultatul obținut pe DATASET_MNIST_PROCESAT indică faptul că modelul se comportă bine pe imaginile proprii preprocesate, apropiate de cele generate prin aplicația practică.

6 IMPLEMENTAREA ACCELERATORULUI HARDWARE

Acest capitol prezintă implementarea acceleratorului hardware în Vitis HLS. Modelul CNN antrenat în software este transpus într-o descriere C/C++ sintetizabilă, utilizând parametrii obținuți în etapa de antrenare. Acceleratorul rezultat execută inferența rețelei neuronale și poate fi generat ca IP hardware pentru integrarea în sistemul embedded bazat pe ZedBoard. În continuare sunt prezentate organizarea proiectului HLS, analiza reprezentării fixed-point, implementarea funcției de inferență, interfațarea prin AXI Stream și optimizările aplicate. Rezultatele simulării, sintezei și comparațiile experimentale sunt prezentate în Capitolul 8.

6.1 ORGANIZAREA IMPLEMENTĂRII ÎN VITIS HLS

Implementarea în Vitis HLS a pornit de la modelul CNN antrenat în software. Straturile definite în Keras au fost rescrise în C/C++ folosind tablouri statice, bucle imbricate și operații aritmetice explicite.

Arhitectura rețelei a fost păstrată, însă implementarea a fost adaptată pentru sinteza hardware. Greutățile și bias-urile obținute după antrenare au fost exportate și incluse în proiectul HLS sub formă de constante.

Implementarea a fost împărțită în două funcții:

- `cnn_forward` realizează calculele rețelei neuronale
- `cnn_axis` asigură transferul datelor prin interfața AXI Stream

Organizarea fișierelor proiectului este următoarea:

Tabelul 6.1. Organizarea fișierelor proiectului HLS.

Fișier	Rol în implementare
<code>cnn.h</code>	definește dimensiunile rețelei, tipurile de date și prototipurile funcțiilor
<code>cnn.cpp</code>	conține funcția <code>cnn_forward</code>
<code>cnn_weights.h</code>	conține greutatea și bias-urile exportate din modelul antrenat
<code>cnn_axis.h</code>	definește tipul de date utilizat pentru transferul prin AXI Stream
<code>cnn_axis.cpp</code>	conține funcția <code>cnn_axis</code> , funcție de nivel superior pentru accelerator
<code>testbench.cpp</code>	transmite imaginile MNIST prin AXI Stream și calculează acuratețea prin argmax

Separarea în fișiere dedicate permite testarea separată a nucleului de calcul și a interfeței de transfer. Funcția `cnn_forward` conține logica rețelei, iar funcția `cnn_axis` adaptează datele la interfața AXI Stream.

Pentru a evidenția modul în care codul C/C++ este transformat într-o implementare hardware, fluxul general de dezvoltare în Vitis HLS este prezentat în Figura 6.1. Această metodologie pornește de la descrierea algoritmului în C/C++, împreună cu testbench-ul și directivele HLS, apoi parcurge etapele de simulare, sinteză, verificare RTL și export al acceleratorului ca IP hardware.

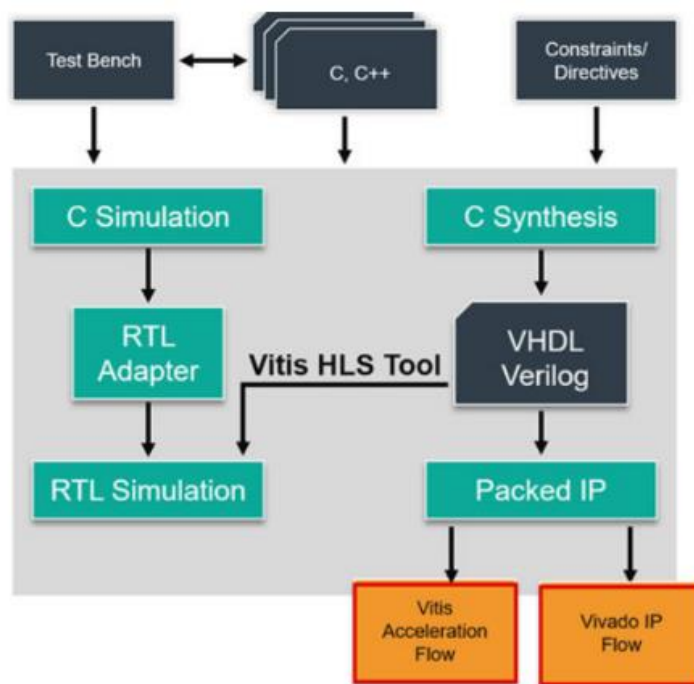


Figura 6.1. Fluxul de dezvoltare în Vitis HLS [5].

Figura 6.1 prezintă fluxul de dezvoltare specific Vitis HLS. În prima etapă, codul C/C++ este verificat prin C Simulation, folosind un fișier de testbench. În cadrul proiectului, această etapă a fost utilizată pentru verificarea funcționării acceleratorului CNN înainte de generarea hardware.

După validarea funcțională, sunt aplicate directivele și constrângerile HLS, iar prin etapa de C Synthesis codul C/C++ este transformat într-o descriere RTL. Această descriere poate fi generată în limbaje hardware precum Verilog sau VHDL. Etapa este importantă deoarece marchează trecerea de la descrierea algoritmică a rețelei CNN la o implementare hardware sintetizabilă.

În continuare, implementarea RTL poate fi verificată prin RTL Simulation sau C/RTL Co-Simulation, pentru a confirma că rezultatul hardware păstrează comportamentul codului C/C++ inițial. La final, acceleratorul este împachetat ca IP hardware, care poate fi importat în Vivado și conectat ulterior în sistemul Zynq împreună cu procesorul ARM, AXI DMA și celelalte componente ale platformei ZedBoard.

6.2 ANALIZA REPREZENTĂRII FIXED-POINT

Modelul a fost antrenat în mediul software folosind reprezentarea float32. Deși această reprezentare oferă precizie ridicată, implementarea operațiilor în virgulă mobilă pe FPGA poate necesita mai multe resurse hardware. Din acest motiv, pentru accelerator a fost analizată utilizarea reprezentării fixed-point. Scopul analizei a fost alegerea unui format numeric care să păstreze acuratețea cât mai aproape de referința software și, în același timp, să fie potrivit pentru implementarea hardware. Testarea variantelor fixed-point a fost realizată prin C Simulation în Vitis HLS, folosind setul MNIST test, format din 10.000 de imagini.

Pentru reprezentarea valorilor în virgulă fixă a fost utilizat tipul de date `ap_fixed` din Vitis HLS, în forma:

```
ap_fixed<W, I, AP_RND, AP_SAT>
```

Codul 6.1. Forma generală a tipului de date `ap_fixed` utilizat în Vitis HLS.

Parametrii utilizați în Codul 6.1 sunt prezentați în Tabelul 6.2.

Tabelul 6.2. Parametrii tipului de date `ap_fixed`.

Parametru	Semnificație
W	numărul total de biți ai valorii reprezentate
I	numărul de biți alocați părții întregi, incluzând bitul de semn
W - I	numărul de biți alocați părții fracționare
AP_RND	rotunjirea valorilor la cea mai apropiată valoare reprezentabilă
AP_SAT	limitarea valorilor care depășesc intervalul numeric permis

Numărul de biți ai părții fracționare poate fi determinat cu relația:

$$F = W - I \quad (6)$$

unde F reprezintă numărul de biți fracționari.

De exemplu, formatul `ap_fixed<8,4>` utilizează 8 biți în total. Dintre aceștia, 4 biți sunt alocați părții întregi, incluzând bitul de semn, iar ceilalți 4 biți sunt alocați părții fracționare. Deoarece există 4 biți fracționari, valoarea reală este scalată cu $2^4 = 16$. Astfel, valoarea 1.0 este stocată intern ca numărul întreg 16, valoarea 0.5 ca numărul întreg 8, iar valoarea 0 este stocată ca 0. În acest mod, numerele fracționare pot fi reprezentate folosind valori întregi pe un număr fix de biți.

Alegerea formatului fixed-point influențează atât acuratețea clasificării, cât și resursele hardware necesare implementării. Un număr prea mic de biți poate duce la pierderi de precizie sau la saturații, în timp ce un număr prea mare de biți poate crește consumul de resurse.

Pentru accelerator a fost utilizată o reprezentare fixed-point cu precizie mixtă. În această abordare, intrările și greutatea sunt reprezentate printr-un format compact, iar activările intermediare, acumulările și scorurile finale folosesc formate mai largi. Această împărțire este necesară deoarece valorile din rețea nu au același rol. De exemplu, acumulările interne pot avea nevoie de un domeniu numeric mai mare, deoarece sunt obținute prin însumarea mai multor produse.

Tipurile fixed-point ale Tipul out_t acceleratorului sunt prezentate în Codul 6.2.

```
// Intrări și greutate  
typedef ap_fixed<8, 4, AP_RND, AP_SAT> data_t;  
// Activări intermediare după straturile convoluționale și pooling  
typedef ap_fixed<10, 5, AP_RND, AP_SAT> act_t;  
// ieșirea stratului FC1  
typedef ap_fixed<10, 6, AP_RND, AP_SAT> fc_t;  
// Acumulări interne pentru convoluții și straturi fully-connected  
typedef ap_fixed<12, 7, AP_RND, AP_SAT> acc_t;  
// Scoruri finale  
typedef ap_fixed<11, 6, AP_RND, AP_SAT> out_t;
```

Codul 6.2. Tipurile fixed-point utilizate în varianta finală a acceleratorului.

Tipul data_t este utilizat pentru intrări și greutate, deoarece acestea pot fi reprezentate printr-un format compact. Tipul act_t este utilizat pentru activările intermediare rezultate după straturile convoluționale și pooling. Pentru ieșirea stratului FC1 a fost definit tipul fc_t, iar pentru acumulările interne a fost utilizat tipul acc_t, cu o lățime mai mare. Această alegere reduce riscul de pierdere a preciziei în timpul operațiilor de înmulțire-acumulare. Tipul out_t este utilizat pentru scorurile finale ale celor 10 clase. Prin utilizarea acestor tipuri, formatele mai largi sunt folosite doar acolo unde este necesar un domeniu numeric mai mare sau o precizie suplimentară, iar formatul compact ap_fixed<8,4> este păstrat pentru intrări și greutate. Variantele fixed-point analizate și comparația acurateților obținute sunt prezentate în capitolul dedicat rezultatelor.

6.3 IMPLEMENTAREA FUNCȚIEI DE INFERENȚĂ

Nucleul de calcul al rețelei este implementat în funcția cnn_forward, care realizează inferența completă a modelului CNN pornind de la imaginea de intrare de 28×28 pixeli și producând 10 scoruri corespunzătoare claselor MNIST. Alegerea clasei prezise se face ulterior prin operația *argmax*, în afara acestei funcții.

Funcția are următorul prototip:

```
void cnn_forward(  
    data_t input[INPUT_H][INPUT_W], // imaginea de intrare  $28 \times 28$   
    out_t output[FC2_OUT]           // cele 10 scoruri finale  
);
```

Codul 6.3. Prototipul funcției cnn_forward.

Structura internă a funcției respectă arhitectura modelului CNN antrenat în software. Datele sunt prelucrate succesiv prin straturile convoluționale, operațiile de pooling, operația de vectorizare și straturile fully-connected. Fluxul de calcul este prezentat în Tabelul 6.3.

Tabel 6.3. Fluxul de calcul implementat în funcția cnn_forward.

Etapă	Operație	Dimensiune rezultat	Tip principal
Intrare	imagine normalizată	28 x 28 x 1	data_t
Conv1 + ReLU	4 filtre convoluționale 3 x 3	26 x 26 x 4	act_t
MaxPool1	reducere cu fereastră 2 x 2	13 x 13 x 4	act_t
Conv2 + ReLU	8 filtre convoluționale 3 x 3	11 x 11 x 8	act_t
MaxPool2	reducere cu fereastră 2 x 2	5 x 5 x 8	act_t
Flatten	transformare în vector	200	act_t
FC1 + ReLU	strat fully-connected cu 32 neuroni	32	fc_t
FC2	strat fully-connected final	10	out_t

Rezultatele fiecărei etape sunt reținute în tablouri statice declarate la începutul funcției. Aceste tablouri reprezintă hărțile de caracteristici ale rețelei, vectorul rezultat după operația Flatten și activările stratului FC1. Ieșirea unui strat devine intrarea stratului următor.

```
act_t conv1_out[CONV1_OUT_H][CONV1_OUT_W][CONV1_OUT_C]; // hărți Conv1: 26x26x4
act_t pool1_out[POOL1_OUT_H][POOL1_OUT_W][POOL1_OUT_C]; // hărți Pool1: 13x13x4
act_t conv2_out[CONV2_OUT_H][CONV2_OUT_W][CONV2_OUT_C]; // hărți Conv2: 11x11x8
act_t pool2_out[POOL2_OUT_H][POOL2_OUT_W][POOL2_OUT_C]; // hărți Pool2: 5x5x8
act_t flat[FLAT_SIZE]; // vector de activări după Flatten: 200 valori
fc_t fc1_out[FC1_OUT]; // activările stratului FC1: 32 valori
```

Codul 6.4. Bufferele intermediare utilizate în funcția cnn_forward.

Greutățile exportate din Keras sunt stocate în tablouri liniare, în ordinea (KH, KW, IN_C, OUT_C) pentru straturile convoluționale și (IN, OUT) pentru cele dense. Accesul corect la aceste tablouri se face prin funcții de indexare dedicate fiecărui strat.

Exemplu pentru indexarea greutăților din primul strat convoluțional:

```
// Conv1: kernel 3x3, 1 canal intrare, 4 filtre de ieșire
static int idx_conv1(int kh, int kw, int ic, int oc) {
    return ((kh * CONV1_K + kw) * CONV1_IN_C + ic) * CONV1_OUT_C + oc;
}
// FC1: 200 intrări, 32 neuroni de ieșire
static int idx_fc1(int i, int o) { return i * FC1_OUT + o; }
// analog: idx_conv2, idx_fc2
```

Codul 6.5. Exemple de funcții de indexare pentru greutățile Conv1 și FC1.

Pentru aplicarea funcției de activare ReLU au fost definite două funcții auxiliare. Prima funcție convertește rezultatul acumulării către tipul act_t, utilizat pentru activările intermediare, iar a doua convertește rezultatul către tipul fc_t, utilizat la ieșirea stratului FC1.

Implementarea funcției ReLU este prezentată în Codul 6.6 .

```
static act_t relu_acc_to_act(act_t x) {
    return (x > 0) ? (act_t)x : (act_t)0; // păstrează valorile pozitive
}
static fc_t relu_acc_to_fc(act_t x) {
    return (x > 0) ? (fc_t)x : (fc_t)0; // elimină valorile negative
}
```

Codul 6.6. Funcțiile ReLU utilizate în implementarea CNN.

Primul strat convoluțional aplică 4 filtre de 3×3 pixeli direct asupra imaginii de intrare, fără padding. Aceasta înseamnă că filtrul nu poate fi centrat pe pixelii de la marginea imaginii, reducând dimensiunea spațială cu $K - 1 = 2$ pe fiecare axă — de la 28×28 la 26×26 . Rezultatul este o hartă de caracteristici de dimensiune $26 \times 26 \times 4$, unde fiecare din cele 4 planuri corespunde unui filtru și codifică un tip diferit de pattern detectat în imagine.

Pentru fiecare poziție spațială (i, j) și fiecare filtru oc , se parcurge fereastra receptivă de 3×3 pixeli, se acumulează produsele pixel–greutate, se adaugă bias-ul și se aplică ReLU. Directiva PIPELINE II=1 instruește sintetizatorul HLS să producă o nouă valoare de ieșire la fiecare ciclu de ceas, prin suprapunerea în timp a iterațiilor.

Calculul unei valori de ieșire în stratul Conv1 este prezentat în Codul 6.7.

```
for (int i = 0; i < CONV1_OUT_H; i++)
    for (int j = 0; j < CONV1_OUT_W; j++)
        for (int oc = 0; oc < CONV1_OUT_C; oc++) {
#pragma HLS PIPELINE II=1
            acc_t sum = (acc_t)conv1_b[oc]; // inițializare cu bias
            for (int kh = 0; kh < CONV1_K; kh++)
                for (int kw = 0; kw < CONV1_K; kw++) {
                    sum += (acc_t)input[i+kh][j+kw]
                        * (acc_t)conv1_w[idx_conv1(kh,kw,0,oc)]; // produs pixel-greutate
                }
            conv1_out[i][j][oc] = relu_acc_to_act(sum); // ReLU și scriere hartă
        }
}
```

Codul 6.7. Implementarea stratului Conv1.

Conv2 urmează același șablon, cu deosebirea că operează pe cele 4 hărți de caracteristici produse de MaxPool1, nu direct pe imaginea de intrare. Aceasta adaugă o buclă suplimentară pe dimensiunea canalelor de intrare ($CONV2_IN_C = 4$), astfel că fiecare valoare de ieșire a Conv2 acumulează $3 \times 3 \times 4 = 36$ produse, față de 9 în Conv1.

Operația MaxPooling reduce dimensiunea spațială a hărților de caracteristici prin selectarea valorii maxime din fiecare fereastră de 2×2 pixeli. Aceasta înjumătățește dimensiunile de înălțime și lățime de la 26×26 la 13×13 după MaxPool1, respectiv de la 11×11 la 5×5 după MaxPool2 păstrând numărul de hărți neschimbat.

Efectul util al MaxPooling este reducerea costului computațional al straturilor următoare și introducerea unei invarianțe locale la translații mici ale imaginii:

```
for (int i = 0; i < POOL1_OUT_H; i++)
    for (int j = 0; j < POOL1_OUT_W; j++)
        for (int c = 0; c < POOL1_OUT_C; c++) {
#pragma HLS PIPELINE II=1
            act_t max_val = conv1_out[i*2][j*2][c];          // primul element din fereastră
            for (int pi = 0; pi < POOL1_K; pi++)
                for (int pj = 0; pj < POOL1_K; pj++) {
                    act_t val = conv1_out[i*2+pi][j*2+pj][c];
                    if (val > max_val) max_val = val;          // actualizare maxim local
                }
            pool1_out[i][j][c] = max_val;
        }
    }
```

Codul 6.8. Implementarea operației MaxPool1.

Înainte de straturile fully-connected, tensorul tridimensional $5 \times 5 \times 8$ rezultat după Pool2 trebuie transformat într-un vector liniar. Operația Flatten copiază elementele în ordinea (H, W, C), consistentă cu ordinea din Keras, producând un vector de $5 \times 5 \times 8 = 200$ de activări:

```
int index = 0;
for (int i = 0; i < POOL2_OUT_H; i++)
    for (int j = 0; j < POOL2_OUT_W; j++)
        for (int c = 0; c < POOL2_OUT_C; c++) {
#pragma HLS PIPELINE II=1
            flat[index++] = pool2_out[i][j][c];
        }
    }
```

Codul 6.9. Transformarea rezultatului Pool2 în vectorul flat.

Stratul FC1 calculează pentru fiecare din cei 32 de neuroni produsul scalar dintre vectorul flat și coloana corespunzătoare din matricea de greutate, adaugă bias-ul și aplică ReLU. Implementarea optimizată a acestui strat este detaliată în Secțiunea 6.5.

Stratul FC2 transformă cele 32 de activări ale FC1 în cele 10 scoruri finale. Spre deosebire de FC1, nu se aplică ReLU — scorurile pot fi negative și sunt folosite direct în operația *argmax*. Un scor mai mare indică o probabilitate mai ridicată ca imaginea să aparțină clasei respective:

```
for (int o = 0; o < FC2_OUT; o++) {
    acc_t sum = (acc_t)fc2_b[o];
    for (int i = 0; i < FC1_OUT; i++) {
#pragma HLS PIPELINE II=1
        sum += (acc_t)fc1_out[i] * (acc_t)fc2_w[idx_fc2(i,o)];
    }
    output[o] = (out_t)sum; // conversie finală la out_t, fără ReLU
}
}
```

Codul 6.10. Calculul scorurilor finale în stratul FC2.

Prin această organizare, `cnn_forward` conține exclusiv logica de inferență a rețelei CNN, fără nicio dependență de interfața hardware. Comunicarea cu exteriorul este tratată separat în funcția `cnn_axis`, descrisă în secțiunea următoare.

6.4 INTERFAȚAREA ACCELERATORULUI PRIN AXI STREAM

Pentru a integra acceleratorul CNN în sistemul Zynq, este necesară o interfață prin care procesorul ARM să poată trimite imagini și să primească rezultatele clasificării. Această interfață este implementată în funcția `cnn_axis`, care folosește protocolul AXI Stream (*Advanced eXtensible Interface Streaming*).

Prototipul funcției este:

```
void cnn_axis(  
    hls::stream<axis_t>& in_stream,    // flux intrare: 784 pixeli  
    hls::stream<axis_t>& out_stream    // flux ieșire: 10 scoruri  
);  
  
#pragma HLS INTERFACE axis    port=in_stream  
#pragma HLS INTERFACE axis    port=out_stream  
#pragma HLS INTERFACE s_axilite port=return    // registru de control start/done
```

Codul 6.11. Prototipul și directivele funcției `cnn_axis`.

Directivele de interfațare stabilesc modul în care acceleratorul comunică cu restul sistemului. Directivele `axis` aplicate porturilor `in_stream` și `out_stream` generează interfețe AXI Stream pentru transferul datelor de intrare și de ieșire. Directiva `s_axilite` aplicată pe portul `return` generează interfața de control prin care procesorul ARM poate porni acceleratorul și poate verifica finalizarea execuției.

Tipul pachetului AXI Stream utilizat este:

```
typedef ap_axis<32, 0, 0, 0> axis_t;
```

Codul 6.12. Definirea tipului de pachet AXI Stream.

Cei patru parametri specifică: lățimea câmpului TDATA (32 biți) și absența semnalelor opționale TDEST, TID și TUSER (setate la 0). Lățimea de 32 de biți permite transmiterea unui pixel sau a unui scor per tranzacție AXI. Pe parcursul implementării au fost analizate două variante de codificare a datelor în câmpul TDATA: o variantă bazată pe valori float, utilizată în etapa de testare, și o variantă raw fixed-point, utilizată pentru implementarea finală. Diferențele dintre cele două variante sunt prezentate în Tabelul 6.4.

Tabelul 6.4. Variante de transfer prin AXI Stream.

Variantă AXI	Intrare AXI	Ieșire AXI	Scop
AXI float	valori float pe 32 biți	scoruri convertite la float	folosită pentru testare
AXI raw fixed	reprezentare binară data_t	reprezentare binară out_t	folosită pentru varianta finală hardware

În varianta AXI float, fiecare valoare primită pe câmpul TDATA este interpretată ca număr float pe 32 de biți și convertită către tipul intern `data_t`. La ieșire, fiecare scor de tip `out_t` este convertit la float înainte de a fi plasat în TDATA. Această variantă a fost folosită în faza de testare, deoarece permite compararea mai ușoară a rezultatelor acceleratorului cu cele obținute în mediul software.

În varianta AXI raw fixed-point, câmpul TDATA transportă direct biții reprezentării fixed-point. La intrare, cei mai puțini biți ai câmpului TDATA sunt interpretați ca valoare de tip `data_t`, iar la ieșire sunt transmiși biții corespunzători valorii `out_t`. Această variantă elimină conversiile intermediare prin float și este mai potrivită pentru integrarea finală, deoarece păstrează același format numeric utilizat intern de accelerator.

Separarea clară între funcțiile `cnn_axis` și `cnn_forward` permite modificarea protocolului de transfer fără schimbarea logicii de calcul a rețelei. Astfel, `cnn_forward` rămâne responsabilă de inferența CNN, iar `cnn_axis` se ocupă de adaptarea datelor la interfața AXI Stream.

6.5 OPTIMIZĂRI HLS UTILIZATE

După implementarea acceleratorului CNN în Vitis HLS, a fost aplicată o optimizare la nivelul codului C/C++ pentru reducerea latenței. Optimizarea a fost realizată fără modificarea arhitecturii rețelei neuronale, a greutateților antrenate sau a reprezentării numerice fixed-point stabilite anterior. Modificarea se aplică doar asupra modului în care este executat calculul hardware.

Optimizarea a fost aplicată asupra stratului FC1. Acest strat a fost ales deoarece are un număr mare de operații repetitive de înmulțire și acumulare. Calculul se face pe elemente consecutive din vectorul flat și pe greutatețile asociate fiecărui neuron, ceea ce permite reorganizarea buclei interne pentru a calcula mai multe produse în aceeași iterație.

Pentru fiecare neuron din FC1, calculul pornește de la bias-ul neuronului respectiv. Apoi sunt parcurse cele 200 de valori din vectorul flat. Fiecare valoare este înmulțită cu greutatea corespunzătoare, iar rezultatul este adăugat la suma parțială. Pentru neuronul cu indexul o , calculul poate fi scris simplificat astfel:

$$fc1_out[o] = ReLU(b[o] + flat[0] \cdot w[0, o] + flat[1] \cdot w[1, o] + \dots + flat[199] \cdot w[199, o])$$

Prin urmare, pentru un singur neuron sunt necesare 200 de produse și acumulări. Deoarece stratul FC1 are 32 de neuroni, calculul complet al acestui strat presupune $200 \times 32 = 6400$ operații de tip MAC, adică operații de înmulțire-acumulare.

Pentru evidențierea zonei asupra căreia a fost aplicată optimizarea, fluxul dintre operația Flatten și straturile fully-connected este prezentat în Figura 6.2.

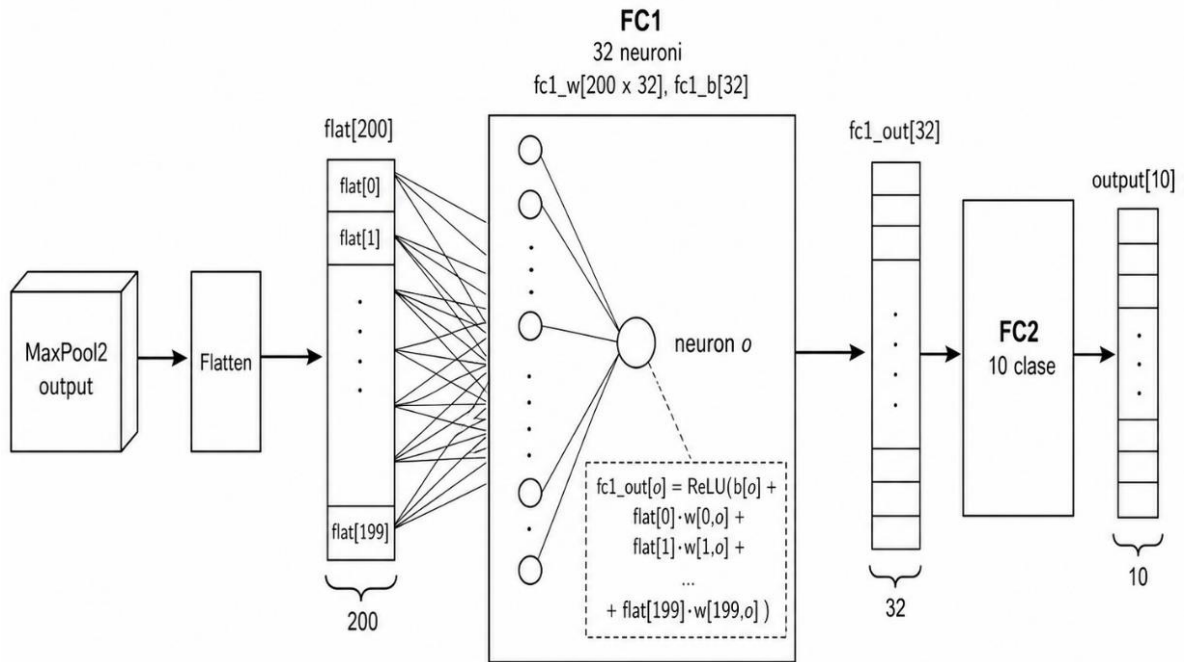


Figura 6.2. Reprezentarea fluxului de date între Flatten, FC1 și FC2.

În varianta de referință, produsele sunt calculate pe rând, deoarece bucla internă parcurge vectorul flat cu pasul 1. În varianta optimizată, vectorul flat și tabloul de greutatea fc1_w sunt partiționați ciclic cu factor 4, iar bucla internă avansează cu pasul 4. Astfel, în aceeași iterație sunt calculate patru produse.

```
#pragma HLS ARRAY_PARTITION variable=flat cyclic factor=4 dim=1
#pragma HLS ARRAY_PARTITION variable=fc1_w cyclic factor=4 dim=1
for (int i = 0; i < FLAT_SIZE; i += 4) {
    #pragma HLS PIPELINE II=1
    sum += (acc_t)flat[i] * (acc_t)fc1_w[idx_fc1(i, o)]
          + (acc_t)flat[i + 1] * (acc_t)fc1_w[idx_fc1(i + 1, o)]
          + (acc_t)flat[i + 2] * (acc_t)fc1_w[idx_fc1(i + 2, o)]
          + (acc_t)flat[i + 3] * (acc_t)fc1_w[idx_fc1(i + 3, o)];
}
```

Codul 6.13. Calculul a patru produse pe iterație prin partiționare ciclică cu factor 4.

Prin această modificare, numărul de iterații necesare pentru calculul unui neuron din FC1 scade de la 200 la 50. Rezultatul matematic nu se modifică, deoarece sunt folosite aceleași valori din vectorul flat și aceleași greutatea asociate neuronului curent. Diferența este dată doar de modul de execuție hardware, operațiile fiind grupate câte patru pentru reducerea latenței.

7 INTEGRAREA SISTEMULUI EMBEDDED

Capitolul prezintă integrarea acceleratorului CNN pe platforma ZedBoard. După generarea IP-ului hardware din Vitis HLS, acceleratorul a fost importat în Vivado și conectat la sistemul Zynq prin AXI DMA. Procesorul ARM controlează comunicația cu aplicația PC, configurează transferurile DMA și pornește acceleratorul, iar logica programabilă execută inferența CNN. Aplicația embedded fpga_server, rulată pe procesorul ARM, realizează legătura dintre aplicația PC și acceleratorul hardware.

7.1 INTEGRAREA ACCELERATORULUI CNN ÎN SISTEMUL HARDWARE

IP-ul acceleratorului CNN generat în Vitis HLS a fost importat în Vivado și integrat în sistemul hardware Zynq. În sistemul final, acceleratorul este implementat în logica programabilă, iar procesorul ARM din Processing System are rol de control și comunicație.

Acceleratorul CNN este conectat la sistem prin trei interfețe principale:

- interfața in_stream, utilizată pentru primirea imaginii de intrare;
- interfața out_stream, utilizată pentru transmiterea scorurilor de clasificare;
- interfața s_axi_control, de tip AXI-Lite, permite procesorului ARM să pornească acceleratorul și să verifice starea acestuia.

Transferul datelor dintre memoria DDR și accelerator este realizat prin modulul AXI DMA, utilizat în modul simplu.

- canalul MM2S, Memory-Mapped to Stream, citește datele din memoria DDR și le transmite către interfața in_stream a acceleratorului
- canalul S2MM, Stream to Memory-Mapped, primește datele de la interfața out_stream și le scrie înapoi în memoria DDR.

Prin această organizare, procesorul ARM nu transmite manual pixelii către accelerator. Procesorul configurează perifericele, stabilește adresele și lungimile transferurilor, iar AXI DMA realizează mutarea efectivă a datelor între DDR și accelerator.

În schema hardware realizată în Vivado, acceleratorul CNN este tratat ca un periferic hardware conectat la sistemul Zynq. Interfața AXI-Lite este folosită pentru semnalele de control, iar interfețele AXI Stream sunt folosite pentru fluxurile de date de intrare și de ieșire. Modulul AXI DMA are rolul de a face legătura între memoria DDR, accesibilă procesorului ARM, și porturile de tip stream ale acceleratorului.

Maparea adreselor utilizate de fpga_server este prezentată în Figura 7.2.

Component	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/axi_dma_0					
/processing_system7_0/S_AXI_HP0	S_AXI_HP0	HP0_DDR_LOWOCM	0x0	512M	0x1FFF_FFFF
/processing_system7_0/S_AXI_HP0	S_AXI_HP0	HP0_DDR_LOWOCM	0x0	512M	0x1FFF_FFFF
Network 1					
/processing_system7_0					
/cnn_axis_0/s_axi_control	s_axi_control	Reg	0x4000_0000	64K	0x4000_FFFF
/axi_dma_0/S_AXI_LITE	S_AXI_LITE	Reg	0x41E0_0000	64K	0x41E0_FFFF

Figura 7.2. Maparea adreselor pentru acceleratorul CNN și AXI DMA în Vivado.

Figura 7.2 evidențiază cele două categorii principale de adrese utilizate în sistem: zona DDR accesibilă prin portul S_AXI_HP0 al sistemului Zynq, mapată în intervalul 0x0000_0000 – 0x1FFF_FFFF, și registrele AXI-Lite ale perifericelor. Prin zona DDR, modulul AXI DMA poate citi imaginea din bufferul de intrare și poate scrie scorurile obținute în bufferul de ieșire. Interfața de control a acceleratorului cnn_axis_0/s_axi_control este mapată la adresa 0x4000_0000, iar interfața de control a modului axi_dma_0/S_AXI_LITE este mapată la adresa 0x41E0_0000. Aceste adrese sunt utilizate în aplicația fpga_server pentru pornirea acceleratorului, configurarea modului DMA și verificarea stării transferurilor. Bufferele folosite de aplicație pentru datele de intrare și de ieșire sunt alocate în zona DDR accesibilă modului DMA, astfel încât canalul MM2S să poată citi imaginea de intrare, iar canalul S2MM să poată scrie scorurile returnate de accelerator în bufferul de ieșire.

Registre AXI DMA utilizate în aplicație sunt prezentate în Tabelul 7.1.

Tabelul 7.1. Registre principale utilizate pentru controlul AXI DMA.

Registru	Offset	Canal	Rol
MM2S_DMACR	0x00	MM2S	control canal MM2S
MM2S_DMASR	0x04	MM2S	stare canal MM2S
MM2S_SA	0x18	MM2S	adresa sursă din DDR
MM2S_LENGTH	0x28	MM2S	lungimea transferului de intrare
S2MM_DMACR	0x30	S2MM	control canal S2MM
S2MM_DMASR	0x34	S2MM	stare canal S2MM
S2MM_DA	0x48	S2MM	adresa destinație din DDR
S2MM_LENGTH	0x58	S2MM	lungimea transferului de ieșire

Pentru acceleratorul HLS este utilizat registrul de control de la offsetul 0x00: bitul AP_START pornește acceleratorul, AP_DONE indică finalizarea execuției, iar AP_IDLE indică faptul că acceleratorul este liber. Pentru fiecare cifră, aplicația PC transmite către ZedBoard 784 bytes, liniarizate într-un vector uint8_t. Pixelii de fundal sunt codificați cu 0, iar pixelii cifrei cu 16, valoare care reprezintă 1.0 în formatul ap_fixed<8,4>: $1.0 \cdot 2^4 = 16$.

După recepție, aplicația `fpga_server` extinde fiecare valoare de la 8 biți la 32 de biți și o scrie în bufferul de intrare din DDR. Extinderea este necesară deoarece transferul AXI Stream utilizat de DMA este pe 32 de biți.

```
static void expand_net_input_to_dma_buffer(void)
{
    for (int i = 0; i < INPUT_SIZE; i++) {
        // valoarea primită de la PC este pe 8 biți
        // DMA transmite cuvinte de 32 de biți
        in_buf[i] = (uint32_t)net_input[i];
    }
}
```

Codul 7.1 Extinderea valorilor `uint8_t` la `uint32_t` în bufferul de intrare.

Extinderea nu modifică valoarea pixelului. Cei 8 biți inferiori ai fiecărui cuvânt conțin valoarea utilă, iar biții superiori sunt zero. Acceleratorul folosește cei 8 biți inferiori ai fiecărui cuvânt primit prin AXI Stream.

Această conversie explică diferența dintre dimensiunea datelor primite prin TCP/IP și dimensiunea transferului realizat către accelerator. Deși imaginea este transmisă de aplicația PC ca 784 valori pe 8 biți, în memoria DDR fiecare valoare este stocată pe 32 de biți pentru a corespunde lățimii interfeței AXI Stream utilizate în sistem. Astfel, conținutul numeric al pixelilor rămâne același, dar spațiul ocupat în bufferul DMA crește. Prin păstrarea acestei codificări simple, aplicația `embedded` nu mai trebuie să efectueze operații suplimentare de normalizare sau conversie numerică înainte de pornirea acceleratorului. Valorile primite de la PC sunt deja compatibile cu formatul de intrare al rețelei, iar extinderea la 32 de biți are doar rol de adaptare la lățimea transferului prin DMA. Această alegere simplifică aplicația `fpga_server` și reduce riscul apariției unor diferențe între datele generate pe PC și datele procesate în accelerator.

Formatul datelor utilizate în transferul dintre aplicația PC, memoria DDR și accelerator este prezentat în Tabelul 7.2.

Tabelul 7.2. Formatul datelor utilizate în transferul PC–ZedBoard–accelerator.

Etapă	Tip date	Dimensiune	Descriere
PC - ZedBoard	<code>uint8_t[784]</code>	784 bytes	imagine 28×28
Buffer intrare DDR	<code>uint32_t[784]</code>	3136 bytes	fiecare pixel extins la 32 biți
Transfer MM2S	AXI Stream 32 biți	3136 bytes	imagine transmisă către accelerator
Buffer ieșire DDR	<code>uint32_t[10]</code>	40 bytes	cele 10 scoruri returnate
ZedBoard - PC	<code>int32_t</code>	4 bytes	cifra prezisă

Fluxul datelor gestionate de aplicația `fpga_server` este prezentat în Figura 7.3. Schema evidențiază transformarea datelor primite de la aplicația PC, transferul acestora prin AXI DMA către accelerator și returnarea predicției finale.

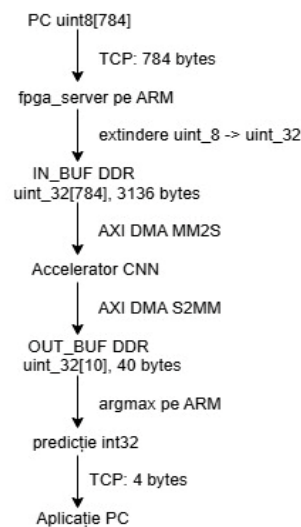


Figura 7.3. Fluxul datelor prin aplicația `fpga_server`, AXI DMA și acceleratorul CNN.

După cum se observă în Figura 7.3, aplicația PC transmite imaginea unei cifre sub forma unui vector de 784 bytes. Aplicația `fpga_server` extinde aceste valori la 32 de biți și le scrie în bufferul de intrare din DDR. Canalul MM2S al modulului AXI DMA transmite datele către acceleratorul CNN, iar canalul S2MM scrie cele 10 scoruri returnate în bufferul de ieșire. Procesorul ARM aplică operația `argmax` și transmite către aplicația PC predicția finală pe 4 bytes.

După pregătirea bufferelor, aplicația `fpga_server` configurează transferurile DMA și pornește acceleratorul. Înainte de transmiterea imaginii către accelerator, este pornit canalul S2MM, astfel încât bufferul de ieșire să fie pregătit pentru scorurile generate. Acceleratorul este pornit după configurarea acestui canal, iar canalul MM2S este pornit ultimul, deoarece acesta transmite imaginea către intrarea AXI Stream.

```

// se setează adresele fizice ale bufferelor din DDR
reg_write(dma_regs, S2MM_DA, OUT_BUF_PHYS);
reg_write(dma_regs, MM2S_SA, IN_BUF_PHYS);
// canalul S2MM este pregătit pentru scorurile de ieșire
reg_write(dma_regs, S2MM_LENGTH, OUTPUT_DMA_BYTES);
// se pornește acceleratorul CNN prin registrul de control
reg_write(hls_regs, HLS_CTRL, HLS_AP_START);
// canalul MM2S trimite imaginea către accelerator
reg_write(dma_regs, MM2S_LENGTH, INPUT_DMA_BYTES);
// se așteaptă finalizarea transferurilor și a inferenței
dma_wait_mm2s_done();
dma_wait_s2mm_done();
hls_wait_done();
  
```

Codul 7.2. Configurarea transferurilor DMA și pornirea acceleratorului CNN.

La ieșire, acceleratorul produce 10 scoruri, corespunzătoare claselor 0–9. Acestea sunt scrise în bufferul de ieșire de canalul S2MM al DMA-ului. Procesorul ARM aplică operația argmax direct pe aceste valori, fără conversie la float, deoarece scorurile au același factor de scalare.

```
static int argmax_fixed_output(volatile uint32_t *buf, int size)
{
    int32_t max_val = (int32_t)buf[0];
    int idx = 0;
    for (int i = 1; i < size; i++) {
        int32_t val = (int32_t)buf[i]; // se păstrează poziția scorului maxim
        if (val > max_val) {
            max_val = val;
            idx = i;
        }
    }
    return idx;
}
```

Codul 7.3. Funcția argmax aplicată pe scorurile din bufferul de ieșire.

Prin această aplicație, procesorul ARM controlează transferul datelor între memoria DDR și acceleratorul CNN, fără a executa inferența propriu-zisă în software. Calculul rețelei neuronale este realizat în logica programabilă, iar aplicația fpga_server gestionează comunicația, transferurile DMA și transmiterea rezultatului final către PC.

7.3 CONCLUZIA CAPITOLULUI

Integrarea realizată permite utilizarea acceleratorului CNN ca periferic hardware controlat de procesorul ARM. Prin conectarea la AXI DMA, datele nu sunt transferate manual de procesor, ci sunt mutate între memoria DDR și accelerator prin canalele MM2S și S2MM.

Aplicația fpga_server asigură controlul execuției pe ZedBoard: primește imaginea de la PC, pregătește datele în formatul necesar transferului DMA, pornește acceleratorul și preia scorurile returnate. Predicția finală este obținută pe procesorul ARM prin operația argmax.

Astfel, capitolul stabilește modul concret în care acceleratorul hardware este folosit în sistemul embedded. După integrarea acestor componente, sistemul poate fi testat practic în fluxul complet PC–ZedBoard–FPGA–PC, aspect prezentat în capitolul următor.

8 REZULTATE EXPERIMENTALE, CONCLUZII ȘI DIRECȚII VIITOARE

Validarea proiectului a fost realizată pe mai multe niveluri, pentru a verifica atât funcționarea acceleratorului CNN, cât și integrarea acestuia în sistemul complet. Testele realizate au urmărit alegerea reprezentării numerice fixed-point, evaluarea optimizărilor HLS, compararea timpului de inferență pe CPU, GPU și FPGA, precum și verificarea funcționării aplicației.

8.1 TESTAREA REPREZENTĂRII NUMERICE FIXED-POINT

Înainte de optimizarea acceleratorului, a fost analizată reprezentarea numerică utilizată pentru calculele interne ale rețelei CNN. Scopul acestei etape a fost reducerea consumului de resurse hardware față de o implementare bazată pe tipuri de date în virgulă mobilă, păstrând în același timp rezultate corecte la clasificare.

Au fost testate mai multe variante fixed-point, diferențiate prin numărul total de biți și prin numărul de biți alocați părții întregi. Pentru fiecare variantă au fost urmărite funcționarea corectă a inferenței și acuratețea obținută pe setul de test MNIST. Rezultatele pentru variantele uniforme sunt prezentate în Tabelul 8.1.

Tabelul 8.1. Acuratețea variantelor fixed-point uniforme pe setul de test MNIST.

Variantă	Format utilizat	Imagini testate	Acuratețe [%]
float	float	10.000	98,33
16_6	ap_fixed<16,6>	10.000	98,31
14_5	ap_fixed<14,5>	10.000	97,51
12_4	ap_fixed<12,4>	10.000	88,43
10_4	ap_fixed<10,4>	10.000	87,96
8_4	ap_fixed<8,4>	10.000	86,69
8_3	ap_fixed<8,3>	10.000	49,06

Pentru toate variantele fixed-point din Tabelul 8.1 au fost păstrate aceleași opțiuni de rotunjire și saturare prezentate anterior. Astfel, valorile rezultate în urma calculelor au fost rotunjite la cea mai apropiată valoare reprezentabilă, iar eventualele depășiri ale intervalului numeric au fost limitate prin saturare.

Pentru o interpretare mai clară a rezultatelor, valorile din Tabelul 8.1 sunt reprezentate în următorul grafic.

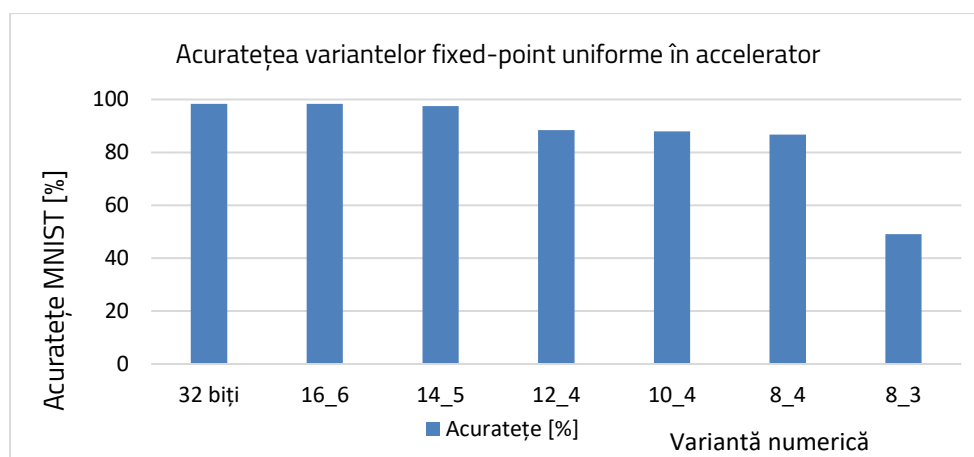


Figura 8.1. Acuratețea variantelor fixed-point uniforme pe setul de test MNIST.

Rezultatele arată că reprezentarea numerică influențează direct acuratețea acceleratorului. Varianta float a fost folosită ca referință, iar varianta 16_6 obține o acuratețe aproape identică, de 98,31%. Formatul 14_5 păstrează de asemenea o acuratețe ridicată, de 97,51%. Pentru formatele 12_4, 10_4 și 8_4, acuratețea scade mai vizibil. Cea mai mare degradare apare pentru 8_3, unde acuratețea ajunge la 49,06%. Această scădere nu înseamnă că toate valorile rețelei trebuie reprezentate pe un număr mare de biți. Problema principală este faptul că, în variantele uniforme, același format numeric este folosit inclusiv pentru acumulările interne. În straturile convoluționale și fully-connected, fiecare valoare de ieșire este obținută prin însumarea mai multor produse. De exemplu, în stratul FC1 fiecare neuron acumulează 200 de produse. Din acest motiv, un format compact precum 8_4 poate fi potrivit pentru intrări și ieșiri, dar poate fi insuficient dacă este folosit și pentru acumulările interne.

În variantele cu precizie mixtă au fost utilizate formate diferite pentru intrări, activări, ieșirea stratului FC1, acumulări și scorurile finale. Această împărțire a fost prezentată în capitolul de implementare, iar în această secțiune sunt analizate rezultatele obținute pentru variantele testate. Rezultatele sunt prezentate în Tabelul 8.2.

Tabelul 8.2. Compararea variantelor fixed-point mixte testate în accelerator.

Variantă	data_t	act_t	fc_t	acc_t	out_t	Acuratețe[%]	Observație
8_4 uniform	8_4	8_4	8_4	8_4	8_4	86,69	Referință uniformă
M1 compactă	8_4	9_5	9_6	10_6	10_6	98,01	variantă mixtă compactă
M2 aleasă	8_4	10_5	10_6	12_7	11_6	98,13	cel mai bun rezultat
M3 mai largă	8_4	12_6	12_7	14_8	12_7	98,11	formate mai largi, fără câștig

Pe baza rezultatelor din Tabelul 8.2, se observă că utilizarea preciziei mixte îmbunătățește semnificativ acuratețea față de varianta uniformă 8_4. Varianta uniformă obține 86,69%, deoarece același format numeric este folosit inclusiv pentru acumulările interne. În aceste acumulări, valorile sunt obținute prin însumarea mai multor produse, iar un format prea compact poate limita atât precizia, cât și domeniul numeric disponibil.

Varianta M1 obține o acuratețe de 98,01%, ceea ce arată că folosirea unor formate mai largi pentru valorile intermediare și acumulări este suficientă pentru a recupera o mare parte din acuratețea pierdută în cazul variantei uniforme. Varianta M2 obține cea mai mare acuratețe dintre variantele mixte testate, 98,13%, și a fost aleasă ca reprezentare numerică finală pentru accelerator.

Varianta M3 utilizează formate mai largi decât M2, dar obține o acuratețe de 98,11%, deci nu aduce o îmbunătățire față de varianta aleasă. Din acest motiv, creșterea suplimentară a numărului de biți nu este justificată. Rezultatul evidențiază faptul că alegerea formatului fixed-point nu depinde doar de numărul total de biți, ci și de modul în care aceștia sunt împărțiți între partea întreagă și partea fracționară. Partea întreagă influențează domeniul numeric al valorilor, iar partea fracționară influențează precizia reprezentării.

8.2 TESTAREA OPTIMIZĂRILOR HLS

Pentru etapa de optimizare au fost testate trei variante, corespunzătoare factorilor de paralelizare 2, 4 și 8. Aceste variante au fost comparate cu implementarea de referință, fără partiționare ciclică suplimentară. Scopul comparației a fost determinarea variantei care oferă cel mai bun compromis între reducerea latenței și consumul de resurse hardware. Valorile analizate au fost latența funcției de top, intervalul de inițiere și resursele utilizate: LUT, FF, BRAM și DSP.

Varianta de referință, notată 00, nu utilizează partiționare ciclică suplimentară asupra calculului din stratul fully-connected. Raportul de sinteză HLS pentru această variantă este prezentat în Figura 8.2.

LATENCY(CYCLES)	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	BRAM	DSP	FF	LUT	URAM
12712	1,530E5	-	12713	-	no	36	3	7794	18820	0
793	9,516E3	-	785	-	loop auto-rew	0	0	386	416	0
11900	1,430E5	-	11900	-	no	36	3	7160	17712	0
12	144.000	-	11	-	loop auto-rew	0	0	19	143	0

Figura 8.2. Raport HLS pentru varianta 00.

Varianta O1 utilizează partiționare ciclică cu factor 2. În acest caz, calculul din stratul fully-connected este reorganizat pentru a procesa două produse în aceeași iterație. Raportul de sinteză HLS este prezentat în Figura 8.3.

LATENCY(CYCLES)	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	BRAM	DSP	FF	LUT	URAM
12712	1,530E5	-	12713	-	no	36	3	7794	18820	0
793	9,516E3	-	785	-	loop auto-rew	0	0	386	416	0
11900	1,430E5	-	11900	-	no	36	3	7160	17712	0
12	144.000	-	11	-	loop auto-rew	0	0	19	143	0

Figura 8.3. Raport HLS pentru varianta O1.

Varianta O2 utilizează partiționare ciclică cu factor 4. Aceasta este varianta aleasă pentru integrarea finală, deoarece urmărește un compromis între reducerea latenței și consumul de resurse. Raportul de sinteză HLS este prezentat în Figura 8.4.

LATENCY(CYCLES)	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	BRAM	DSP	FF	LUT	URAM
7976	9,571E4	-	7977	-	no	40	3	8048	19464	0
793	9,516E3	-	785	-	loop auto-rew	0	0	386	416	0
7164	8,597E4	-	7164	-	no	40	3	7414	18356	0
12	144.000	-	11	-	loop auto-rew	0	0	19	143	0

Figura 8.4. Raport HLS pentru varianta O2.

Varianta O3 utilizează partiționare ciclică cu factor 8. Aceasta oferă cel mai mare grad de paralelizare dintre variantele testate. Raportul de sinteză HLS este prezentat în Figura 8.5.

LATENCY(CYCLES)	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	BRAM	DSP	FF	LUT	URAM
7208	8,650E4	-	7209	-	no	64	3	8207	20343	0
793	9,516E3	-	785	-	loop auto-rew	0	0	386	416	0
6396	7,675E4	-	6396	-	no	64	3	7573	19235	0
12	144.000	-	11	-	loop auto-rew	0	0	19	143	0

Figura 8.5. Raport HLS pentru varianta O3.

Pentru o comparație mai clară între variantele analizate, valorile principale obținute la sinteză au fost sintetizate în Tabelul 8.3. Tabelul prezintă latența totală estimată a acceleratorului și resursele hardware utilizate pentru fiecare variantă.

Tabelul 8.3. Compararea rezultatelor obținute la sinteză pentru variantele de optimizare.

Varianta	Factor de paralelizare	Latență [cicluri]	Interval [cicluri]	LUT	FF	BRAM	DSP
O0	-	12712	12713	18820	7794	36	3
O1	2	9544	9545	19083	7918	36	3
O2	4	7976	7977	19464	8048	40	3
O3	8	7208	7209	20343	8207	64	3

Din Tabelul 8.3 se observă reducerea latenței odată cu aplicarea optimizărilor HLS. Varianta O0 are cea mai mare latență, de 12712 cicluri, deoarece reprezintă implementarea de referință, fără partiționare ciclică suplimentară. Prin aplicarea factorului 2, latența scade la 9544 cicluri, ceea ce reprezintă o reducere de 3168 cicluri, aproximativ 24,9%. Această îmbunătățire este obținută fără creșterea numărului de BRAM, care rămâne egal cu 36. Pentru varianta O2, cu factor 4, latența scade la 7976 cicluri, reducerea față de O0 fiind de 4736 cicluri, adică aproximativ 37,3%. În acest caz, consumul de BRAM crește moderat, de la 36 la 40, iar valorile LUT și FF cresc ușor. Varianta O3, cu factor 8, obține cea mai mică latență, de 7208 cicluri, adică o reducere de 5504 cicluri, aproximativ 43,3%. Totuși, această îmbunătățire vine cu o creștere semnificativă a consumului de BRAM, de la 36 la 64. Reducerea latenței nu este direct proporțională cu factorul de paralelizare, deoarece optimizarea a fost aplicată doar asupra stratului fully-connected analizat, în timp ce latența totală include și celelalte etape ale acceleratorului. Numărul de DSP rămâne constant pentru toate variantele, având valoarea 3, iar creșterea LUT, FF și BRAM este determinată de accesul paralel la date și de logica suplimentară de acumulare. Deși varianta O3 oferă cea mai mică latență, consumul ridicat de BRAM nu justifică alegerea ei pentru implementarea finală. Varianta O2 a fost aleasă deoarece oferă un compromis mai bun între reducerea latenței și resursele hardware utilizate. Pentru evidențierea mai clară a diferențelor de latență dintre variantele analizate, au fost reprezentate grafic în Figura 8.6.

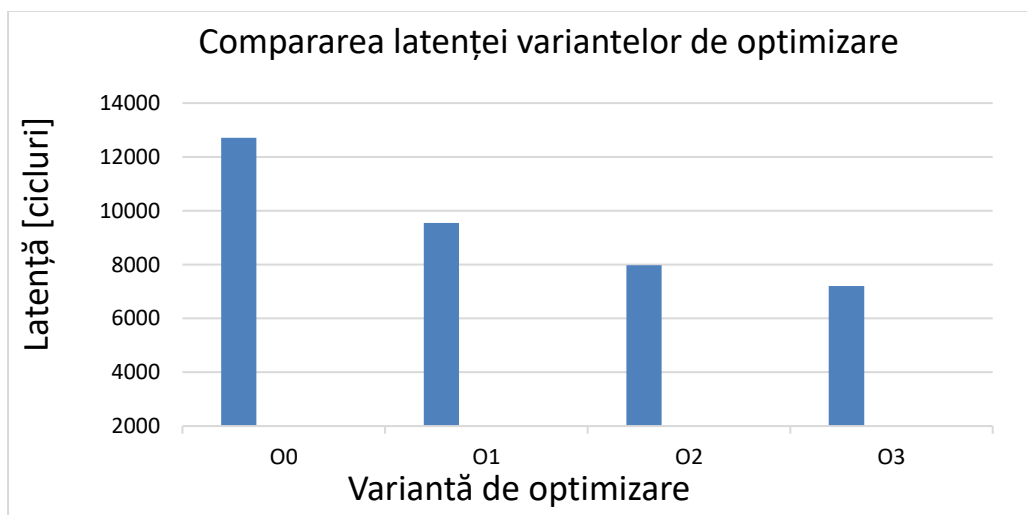


Figura 8.6. Compararea latenței pentru variantele de optimizare HLS.

Graficul evidențiază scăderea latenței odată cu creșterea factorului de paralelizare. Se observă că varianta O0 are latența cea mai mare, iar varianta O3 are latența cea mai mică. Totuși, alegerea variantei finale nu se face doar pe baza latenței, ci și în funcție de resursele hardware utilizate, motiv pentru care varianta O2 a fost considerată cea mai potrivită pentru implementarea finală.

8.3 COMPARAREA TIMPULUI DE INFERENȚĂ CPU, GPU ȘI FPGA

Pentru evaluarea performanței rețelei CNN, a fost realizată o comparație între execuția modelului pe CPU, GPU și FPGA, urmărind timpul mediu necesar pentru clasificarea unei imagini de intrare de dimensiune 28×28 pixeli. Pentru platformele CPU și GPU a fost utilizat modelul CNN antrenat în TensorFlow/Keras, testarea fiind realizată în Google Colab pe același set de 1000 de imagini MNIST. Execuția pe CPU și execuția pe GPU au fost selectate explicit în script, astfel încât măsurătorile să fie realizate separat, în același mediu de testare. Pentru GPU a fost utilizat acceleratorul NVIDIA Tesla T4 disponibil în sesiunea Colab. Timpul raportat pentru CPU și GPU reprezintă timpul mediu de inferență pentru o imagine, fără includerea timpului de încărcare a modelului. Înainte de măsurarea propriu-zisă au fost realizate rulări de încălzire, pentru reducerea influenței inițializărilor interne TensorFlow, iar valorile prezentate reprezintă media a două rulări succesive. Pentru FPGA, timpul de inferență a fost estimat pe baza latenței raportate de Vitis HLS pentru varianta O2 a acceleratorului CNN. Această variantă are o latență de 7976 cicluri. Considerând frecvența de 100 MHz, timpul estimat al acceleratorului este:

$$Timp_{FPGA} = \frac{7976}{100.000.000} = 0,00007976 \text{ s} = 79,76 \mu\text{s} = 0,07976 \text{ ms}$$

Rezultatele comparației sunt prezentate în Tabelul 8.4.

Tabelul 8.4. Compararea timpului de inferență pe CPU, GPU și FPGA.

Platformă	Mediu de testare	Metodă de execuție	Timp mediu / imagine [ms]
CPU	Google Colab	TensorFlow/Keras	8,680
GPU	Google Colab, NVIDIA, Tesla T4	TensorFlow/Keras	8,002
FPGA	Vitis HLS, varianta O2	accelerator CNN	0,07976

Pentru platformele CPU și GPU a fost utilizat modelul CNN antrenat în TensorFlow/Keras, testarea fiind realizată în Google Colab pe același set de 1000 de imagini MNIST. Execuția pe CPU a fost realizată pe procesorul disponibil în sesiunea Google Colab, iar execuția pe GPU a fost realizată pe acceleratorul NVIDIA Tesla T4 disponibil în aceeași platformă. Deoarece resursele oferite de Google Colab pot varia între sesiuni, valorile obținute sunt considerate rezultate experimentale pentru mediul de testare utilizat.

Rezultatele obținute arată că execuția pe GPU este ușor mai rapidă decât execuția pe CPU. Timpul mediu scade de la 8,680 ms pe CPU la 8,002 ms pe GPU. Raportul dintre cele două valori este:

$$\frac{CPU}{FPGA} = \frac{8,680}{0,07976} \approx 1,08$$

Astfel, în testarea imagine cu imagine, GPU-ul obține o accelerare de aproximativ 1,08 ori față de CPU. Diferența este redusă deoarece rețeaua CNN utilizată este compactă, iar fiecare inferență procesează o singură imagine de 28×28 pixeli. În acest caz, overhead-ul introdus de apelurile TensorFlow, transferul datelor către dispozitiv și lansarea operațiilor pe GPU devine comparabil cu timpul efectiv de calcul.

Pentru acceleratorul FPGA, timpul estimat este de 0,07976 ms pentru o imagine. Raportul dintre timpul CPU și timpul FPGA este:

$$\frac{CPU}{FPGA} = \frac{8,680}{0,07976} \approx 108,82$$

Raportul dintre timpul GPU și timpul FPGA este:

$$\frac{GPU}{FPGA} = \frac{8,002}{0,07976} \approx 100,32$$

Rezultatele indică faptul că acceleratorul FPGA reduce semnificativ timpul de inferență pentru o imagine individuală. Această diferență apare deoarece implementarea FPGA este o implementare hardware dedicată, în care operațiile rețelei CNN sunt transpuse direct în logică digitală și organizate prin optimizări HLS. În schimb, execuția pe CPU și GPU se face printr-un framework software general, care introduce costuri suplimentare asociate apelurilor de funcții, planificării operațiilor și sincronizării rezultatelor.

Trebuie menționat că valoarea raportată pentru FPGA reprezintă timpul estimat al acceleratorului, calculat din latența Vitis HLS, și nu timpul complet al sistemului. Aceasta nu include comunicația TCP/IP, pregătirea bufferelor în memoria DDR, configurarea AXI DMA sau operațiile realizate de procesorul ARM. Prin urmare, comparația evidențiază timpul de execuție al inferenței propriu-zise, nu timpul total al fluxului PC–ZedBoard–FPGA–PC.

8.4 VERIFICAREA FUNCȚIONĂRII SISTEMULUI INTEGRAT

Pentru validarea practică a proiectului a fost testat sistemul integrat PC–ZedBoard–FPGA. Testele au urmărit verificarea funcționării aplicației, a comunicației cu placa ZedBoard și a rezultatului returnat de acceleratorul CNN implementat pe FPGA. Înainte de testarea aplicației, a fost verificată conexiunea de rețea între laptop și placa ZedBoard.

Figura 8.7 prezintă configurarea adresei IP pe placa ZedBoard, necesară pentru conectarea acesteia în aceeași rețea cu laptopul pe care rulează aplicația.

```
petalinux@zedboard_cnn:~/app$ sudo ifconfig eth0 192.168.1.10 netmask 255.255.255.0 up
```

Figura 8.7. Configurarea adresei IP pe placa ZedBoard.

Conexiunea dintre laptop și ZedBoard a fost verificată prin comanda *ping*.

```
Pinging 192.168.1.10 with 32 bytes of data:
Reply from 192.168.1.10: bytes=32 time<1ms TTL=64
Reply from 192.168.1.10: bytes=32 time<1ms TTL=64
Reply from 192.168.1.10: bytes=32 time<1ms TTL=64
Reply from 192.168.1.10: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
(.venv) PS E:\camera_fpga_app>
```

Figura 8.8. Verificarea conexiunii dintre PC și ZedBoard prin comanda *ping*.

După verificarea conexiunii, pe placa ZedBoard a fost pornit serverul TCP care așteaptă conexiunea aplicației pe portul configurat.

```
=====
FPGA CNN TCP Server - FIXED 8_4
=====
HLS_BASE_PHYS      = 0x40000000
DMA_BASE_PHYS      = 0x41E00000
IN_BUF_PHYS        = 0x1E000000
OUT_BUF_PHYS        = 0x1E001000
INPUT_NET_BYTES     = 784
INPUT_DMA_BYTES     = 3136
OUTPUT_DMA_BYTES    = 40
SERVER_PORT         = 12345
=====
Server pornit. Astept client pe portul 12345...
```

Figura 8.9. Pornirea serverului TCP pe placa ZedBoard.

Aceste verificări confirmă faptul că placa ZedBoard este accesibilă din aplicația de pe PC și poate primi date prin conexiunea TCP.

8.4.1 Testarea transmiterii unei cifre către accelerator

După pornirea serverului, a fost testată transmiterea unei singure cifre către accelerator. Acest test a avut rolul de a verifica întregul traseu al datelor, începând de la aplicația PC și până la obținerea rezultatului returnat de sistemul FPGA. Cifra detectată în imagine este extrasă din zona de interes, normalizată la dimensiunea 28×28 pixeli și transmisă către placa ZedBoard prin conexiunea TCP/IP.

Figura 8.10 prezintă interfața aplicației după detecția cifrei, normalizarea imaginii și afișarea rezultatului returnat de FPGA. În fereastra aplicației se poate observa imaginea capturată, zona detectată, forma binarizată utilizată pentru inferență și rezultatul final primit de la accelerator.

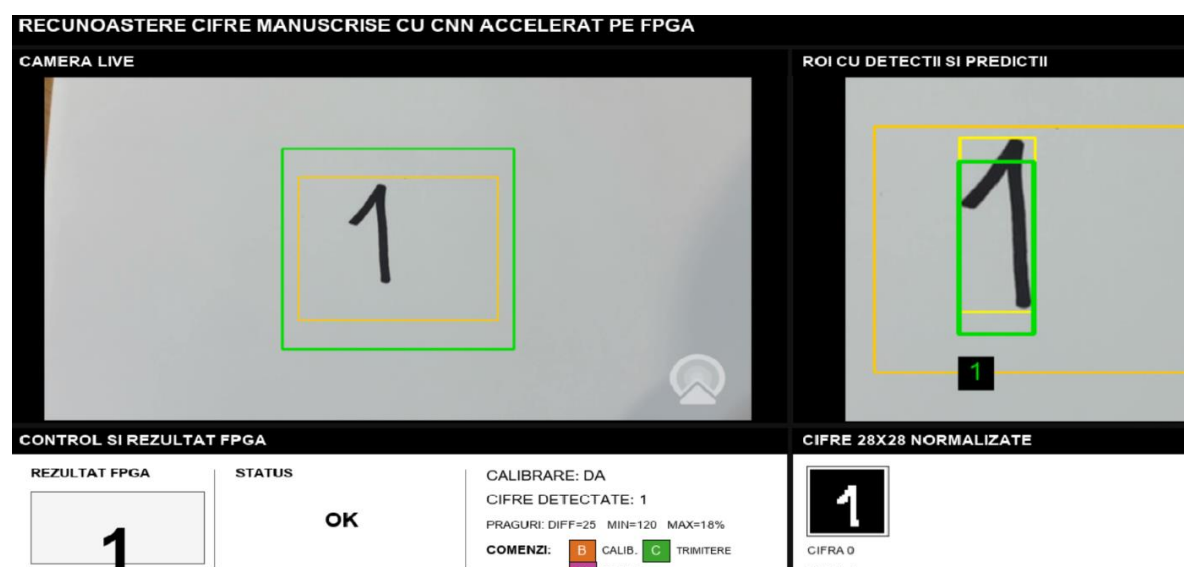


Figura 8.10. Testarea sistemului pentru o singură cifră.

Pentru aceeași rulare, terminalele aplicației și ale plăcii ZedBoard au fost urmărite pentru verificarea schimbului de date. Figura 8.11 confirmă transmiterea datelor către placă și primirea rezultatului calculat de accelerator.

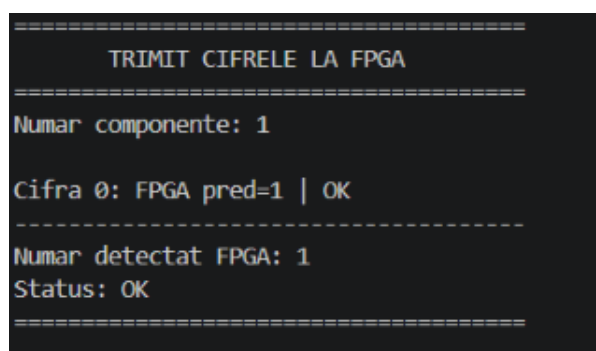


Figura 8.11. Mesaje afișate în terminal în timpul clasificării unei cifre.

În etapa următoare a fost verificată procesarea mai multor cifre aflate în aceeași imagine. Figura 8.12 prezintă un exemplu cu mai multe cifre detectate, normalizate separat și transmise succesiv către accelerator.

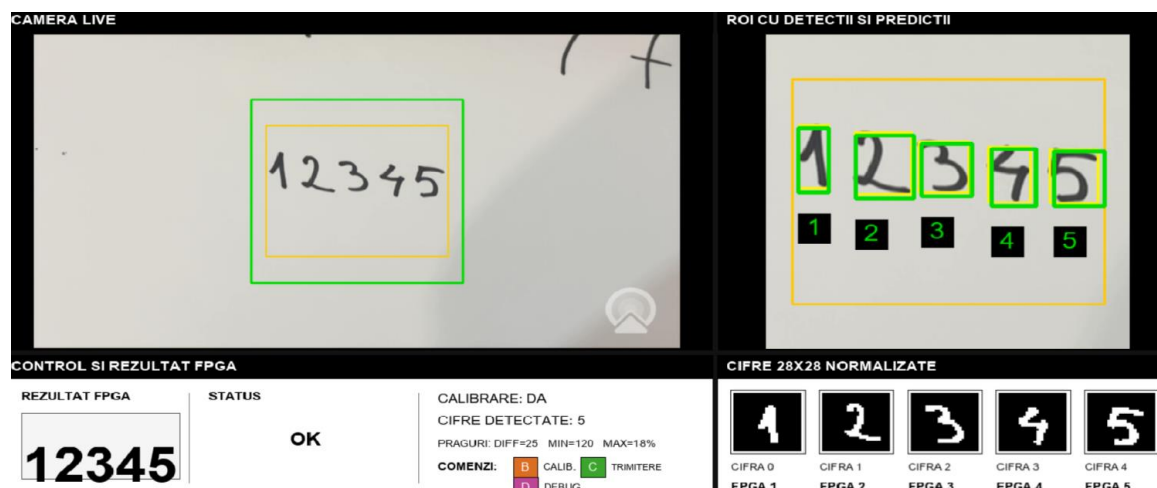


Figura 8.12. Testarea sistemului pentru mai multe cifre.

Rezultatul afișat este format prin concatenarea predicțiilor returnate pentru fiecare cifră detectată. Rezultatul obținut în aplicație pentru această rulare este prezentat în Figura 8.13.

```
Numar componente: 5

Cifra 0: FPGA pred=1 | OK
Cifra 1: FPGA pred=2 | OK
Cifra 2: FPGA pred=3 | OK
Cifra 3: FPGA pred=4 | OK
Cifra 4: FPGA pred=5 | OK

-----
Numar detectat FPGA: 12345
Status: OK
=====
```

Figura 8.13. Rezultatul concatenat afișat pentru mai multe cifre detectate.

Pentru a verifica influența modului de scriere asupra rezultatului, a fost testată aceeași cifră scrisă în două forme diferite. Figura 8.14 prezintă clasificarea cifrei 7 scrise în două moduri diferite.



Figura 8.14. Clasificarea aceleiași cifre scrise în două moduri diferite.

Ambele teste verifică faptul că sistemul poate clasifica aceeași cifră în condiții de scriere diferite, atât timp cât cifra este detectată corect și forma acesteia este păstrată după normalizare.

8.4.2 Testarea unor cazuri particulare

A fost testat și un caz cu o cifră scrisă mai rotunjit și ușor înclinată, pentru a observa comportamentul sistemului în situații mai puțin ideale. Figura 8.15 prezintă detecția, normalizarea și rezultatul obținut.

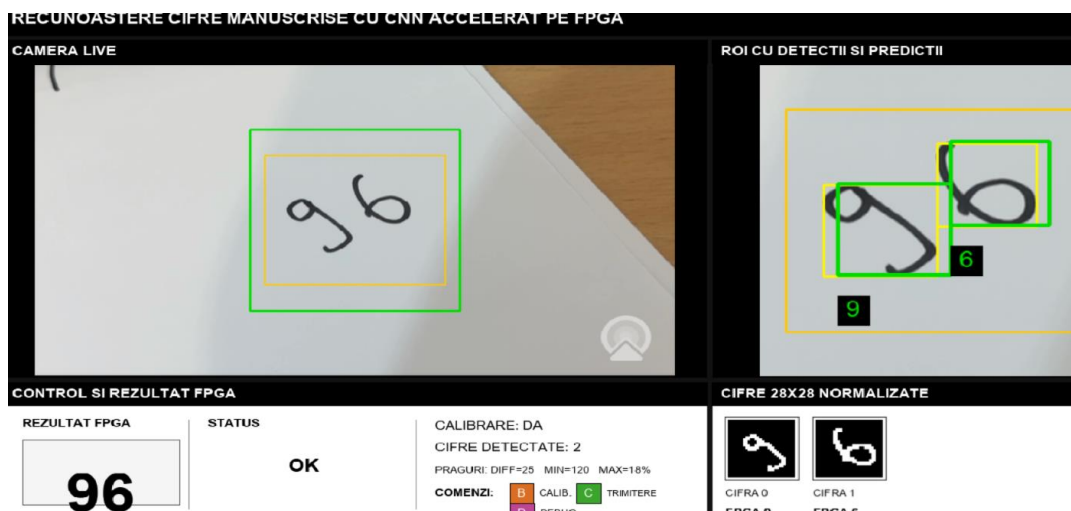


Figura 8.15. Clasificarea unei cifre scrise înclinat.

Rezultatul arată că etapa de detecție și preprocesare poate gestiona forme care nu sunt perfect aliniate, dacă cifra rămâne suficient de clară pentru clasificare.

8.4.3 Limitarea observată: cifre pe verticală

În timpul testării a fost observată și o limitare în cazul cifrelor dispuse vertical. Figura 8.16 prezintă un exemplu în care cifrele sunt detectate, însă ordinea rezultatului nu corespunde ordinii naturale de citire de sus în jos.

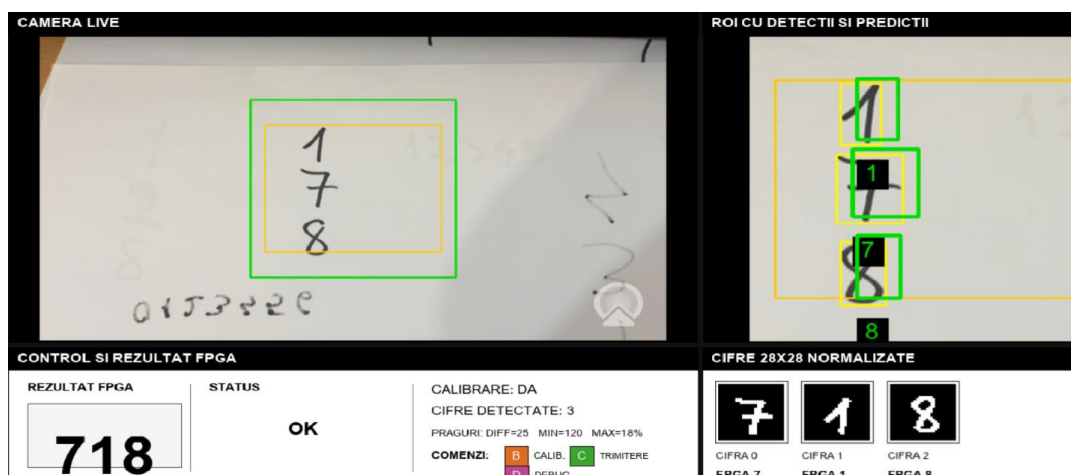


Figura 8.16. Limitare observată pentru cifre dispuse vertical.

8.5 CONCLUZII

În cadrul acestui proiect a fost realizat un sistem embedded pentru recunoașterea cifrelor scrise de mână, utilizând o rețea neuronală convoluțională implementată ca accelerator hardware pe FPGA. Sistemul dezvoltat integrează aplicația PC, conexiunea TCP/IP, procesorul ARM de pe placa ZedBoard, modulul AXI DMA și acceleratorul CNN generat prin Vitis HLS.

Rezultatele obținute au confirmat funcționarea sistemului propus. Rețeaua CNN a fost antrenată și testată inițial în format Float32, după care au fost analizate mai multe variante fixed-point pentru implementarea hardware. Varianta finală a utilizat precizie mixtă, deoarece aceasta a permis păstrarea unei acurateți ridicate, apropiate de cea a modelului software, reducând în același timp complexitatea reprezentării numerice față de o implementare în virgulă mobilă.

Optimizările HLS aplicate asupra acceleratorului au redus latența de execuție față de varianta inițială. Dintre variantele analizate, O2 a fost aleasă ca soluție finală, deoarece a oferit un compromis bun între reducerea latenței și consumul de resurse hardware. Compararea timpului de inferență pe CPU, GPU și FPGA a evidențiat avantajul acceleratorului hardware pentru procesarea unei imagini individuale, timpul estimat al acceleratorului fiind semnificativ mai mic decât cel obținut prin execuție software.

Validarea practică a arătat că aplicația PC poate detecta și preprocesa cifra, poate transmite datele către placa ZedBoard, iar rezultatul calculat de accelerator poate fi returnat și afișat în interfața grafică. Testele realizate au confirmat funcționarea fluxului complet PC–ZedBoard–FPGA–PC, atât pentru cazuri simple, cu o singură cifră, cât și pentru cazuri cu mai multe cifre detectate.

Prin realizarea acestui proiect au fost îndeplinite obiectivele propuse: implementarea unei rețele CNN pentru recunoașterea cifrelor, conversia acesteia într-o formă potrivită pentru FPGA, integrarea acceleratorului într-un sistem embedded și validarea funcționării printr-o aplicație software. Proiectul demonstrează utilitatea platformelor FPGA în aplicații de inferență neurală, în special atunci când este necesară procesarea rapidă a datelor într-un sistem dedicat.

8.6 DIRECȚII VIITOARE

O direcție viitoare de dezvoltare este extinderea sistemului pentru recunoașterea literelor și a caracterelor alfanumerice, nu doar a cifrelor. Prin antrenarea rețelei pe un set de date mai complex, aplicația ar putea fi utilizată pentru interpretarea unor texte scurte, formulare sau documente simple.

O altă îmbunătățire posibilă este adaptarea sistemului pentru imagini obținute de la un scanner sau de la o cameră în condiții mai variate. În acest sens, etapa de preprocesare ar putea fi îmbunătățită pentru a trata fundaluri diferite, umbre, iluminare neuniformă, înclinarea foii și caractere dispuse pe mai multe rânduri.

De asemenea, acceleratorul hardware poate fi optimizat suplimentar prin analizarea altor directive HLS sau prin integrarea unor operații suplimentare direct în FPGA. De exemplu, operația

argmax ar putea fi inclusă în accelerator, astfel încât procesorul ARM să primească direct clasa prezisă.

Prin aceste dezvoltări, proiectul ar putea fi extins într-un sistem embedded pentru recunoașterea automată a caracterelor din imagini capturate sau scanate, util în aplicații precum citirea formularelor, identificarea valorilor numerice de pe etichete sau digitalizarea unor documente simple.

- [1] IBM. What is Computer Vision? <https://www.ibm.com/think/topics/computer-vision>
- [2] LeCun, Y., Bengio, Y., Hinton, G. (2015). Deep learning. Nature, 521, 436–444. <https://doi.org/10.1038/nature14539>
- [3] Gao, L., Luo, Z., Wang, L. (2025). Convolutional Neural Network Acceleration Techniques Based on FPGA Platforms: Principles, Methods, and Challenges. Information, 16(10), 914. <https://doi.org/10.3390/info16100914>
- [4] TensorFlow Datasets. MNIST Dataset. <https://www.tensorflow.org/datasets/catalog/mnist>
- [5] AMD. Vitis High-Level Synthesis User Guide, UG1399. <https://docs.amd.com/r/en-US/ug1399-vitis-hls>
- [6] Digilent. ZedBoard Zynq-7000 Development Board Reference Manual. <https://digilent.com/reference/programmable-logic/zedboard/reference-manual>
- [7] AMD. Vivado Design Suite: AXI Reference Guide, UG1037. <https://docs.amd.com/v/u/en-US/ug1037-vivado-axi-reference-guide>
- [8] AMD. AXI DMA LogiCORE IP Product Guide, PG021. https://docs.amd.com/r/en-US/pg021_axi_dma
- [9] AMD. PetaLinux Tools Documentation: Reference Guide, UG1144. <https://docs.amd.com/r/en-US/ug1144-petalinux-tools-reference-guide>

Lucrarea prezintă proiectarea și implementarea unui sistem embedded pentru recunoașterea cifrelor scrise de mână, utilizând o rețea neuronală convoluțională accelerată hardware pe FPGA. Sistemul propus integrează o aplicație software realizată în Python, placa ZedBoard bazată pe arhitectura Zynq și un accelerator CNN implementat în Vitis HLS.

Aplicația software are rolul de a prelua imaginea de la cameră, de a extrage regiunea de interes, de a aplica etapele de preprocesare și de a separa cifrele detectate. Fiecare cifră este normalizată la dimensiunea de 28×28 pixeli și transmisă către placa ZedBoard printr-o conexiune TCP/IP. Pe partea embedded, aplicația fpga_server rulează sub PetaLinux pe procesorul ARM și gestionează comunicația cu PC-ul, transferurile prin AXI DMA și pornirea acceleratorului hardware.

Modelul CNN a fost antrenat și testat inițial în software, apoi a fost adaptat pentru implementarea hardware prin utilizarea reprezentării fixed-point. Acceleratorul primește imaginea normalizată, execută inferența rețelei neuronale și returnează scorurile corespunzătoare claselor 0–9. Predicția finală este obținută prin alegerea scorului maxim.

Pentru validarea sistemului au fost realizate teste privind acuratețea variantelor fixed-point, optimizările HLS, timpul de inferență și funcționarea fluxului complet PC, ZedBoard, FPGA, PC. Rezultatele obținute confirmă funcționarea sistemului integrat și evidențiază avantajul utilizării FPGA pentru accelerarea inferenței într-o aplicație embedded dedicată.

This diploma project presents the design and implementation of an embedded system for handwritten digit recognition using a convolutional neural network accelerated in FPGA hardware. The proposed system integrates a Python software application, the ZedBoard platform based on the Zynq architecture, and a CNN accelerator developed in Vitis HLS.

The software application captures the image from a camera, extracts the region of interest, applies the preprocessing steps, and separates the detected digits. Each digit is normalized to a 28×28 pixel image and transmitted to the ZedBoard through a TCP/IP connection. On the embedded side, the `fpga_server` application runs under PetaLinux on the ARM processor and manages the communication with the PC, the AXI DMA transfers, and the control of the hardware accelerator.

The CNN model was first trained and evaluated in software, then adapted for hardware implementation using fixed-point representation. The accelerator receives the normalized image, performs the neural network inference, and returns the output scores corresponding to the classes from 0 to 9. The final prediction is obtained by selecting the class with the highest score.

The system was validated through tests related to fixed-point accuracy, HLS optimizations, inference time, and the complete PC–ZedBoard–FPGA–PC data flow. The obtained results confirm the correct operation of the integrated system and highlight the advantage of using FPGA-based acceleration for neural network inference in a dedicated embedded application.